

# 第8章 善于利用指针

---

# 概述

## ■ 指针的用途:

- 方便有效地使用字符串、数组
- 为函数提供了通过形参双向传递数据的途径
- 直接访问内存地址
  - 使程序简洁、紧凑、高效
- 构造复杂的数据结构（§ 9）
- 动态分配内存（§ 9）

## ■ 指针的危险性:

- 使用未初始化的指针可能导致系统瘫痪
- 对指针的错误使用往往难以查找

# 主要内容

---

- 8.1 指针是什么
- 8.2 指针变量
- 8.3 通过指针引用数组 (8.3.5\*)
- 8.4 通过指针引用字符串
- 8.5\* 指向函数的指针
- 8.6\* 返回指针值的函数
- 8.7\* 指针数组和多重指针
- 8.8 动态内存分配与指向它的指针变量
- 8.9 有关指针的小结

# 8.1 指針是什么

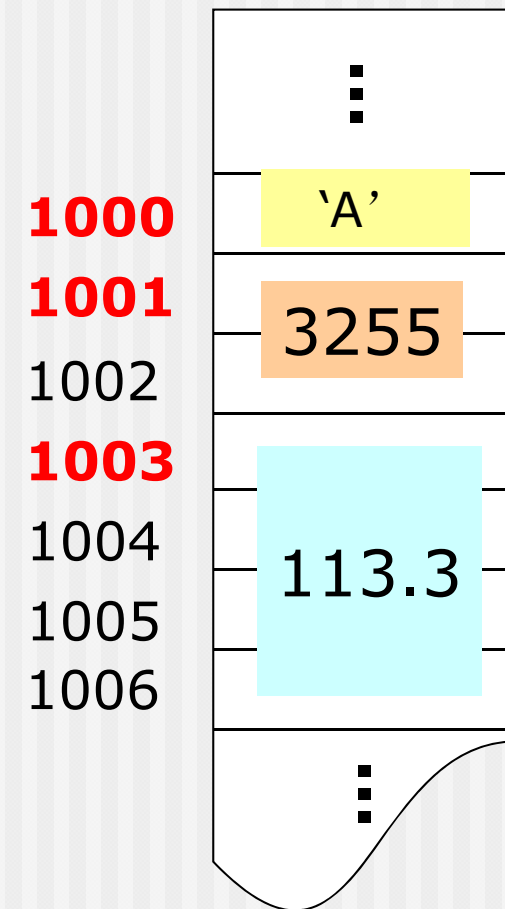
---

## 8.1 指针是什么

### ■ 内存空间

- 内存按字节编址，即给内存中的每个字节一个唯一的编号，这个编号就是“地址”。
- 数据在内存中的存储
  - 根据数据类型分配一定长度的空间作为存储单元
  - 以首字节地址作为该存储单元的地址

内存中的  
用户数据区



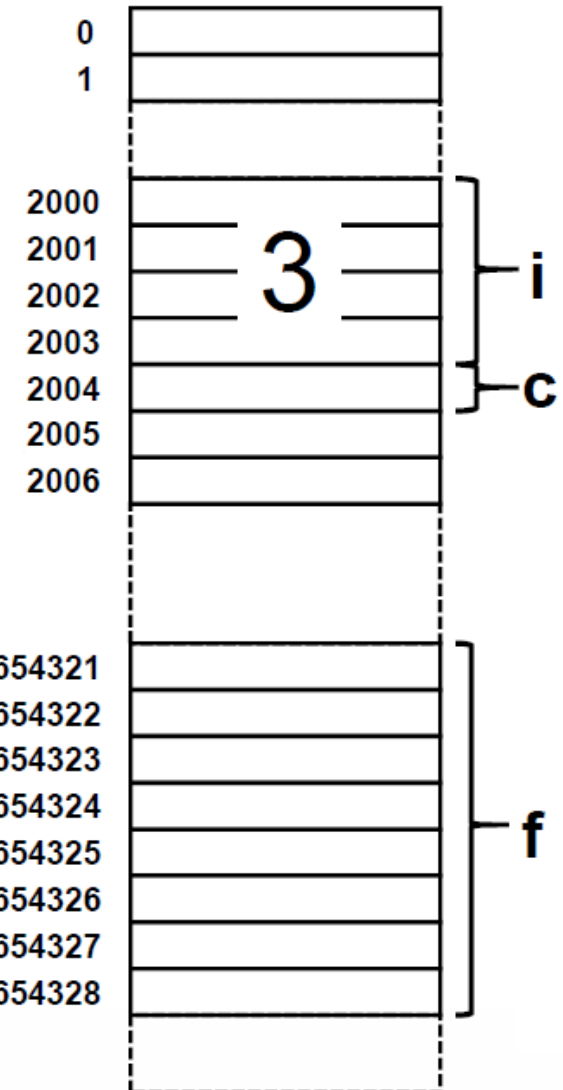
# C语言编译器对内存的使用

```
int i;  
char c;  
double f;  
i = 3;
```

- C语言程序编译运行时，编译器为变量分配存储空间，并建立起变量名与地址之间的对应关系。



| 符号表  |      |      |            |
|------|------|------|------------|
| 变量名  | i    | c    | f          |
| 地址   | 2000 | 2004 | 0x87654321 |
| 数据类型 | int  | char | double     |



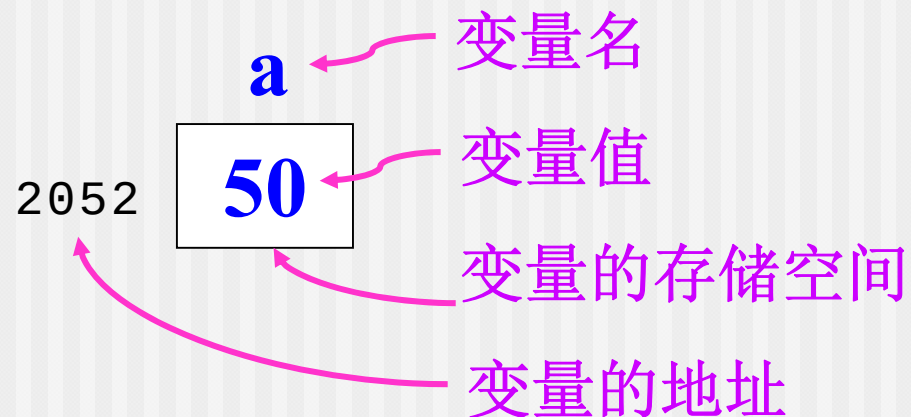


## 8.1 指针是什么（2）

### ■ 内存单元的地址与内存单元的内容

- 在程序中通常使用变量名来直接对数据进行访问
- 变量名实质上是变量地址的符号表示
- 在编译时，变量名将被转换成变量的地址。

地址 —— 变量名 —— 变量的值（/内容）



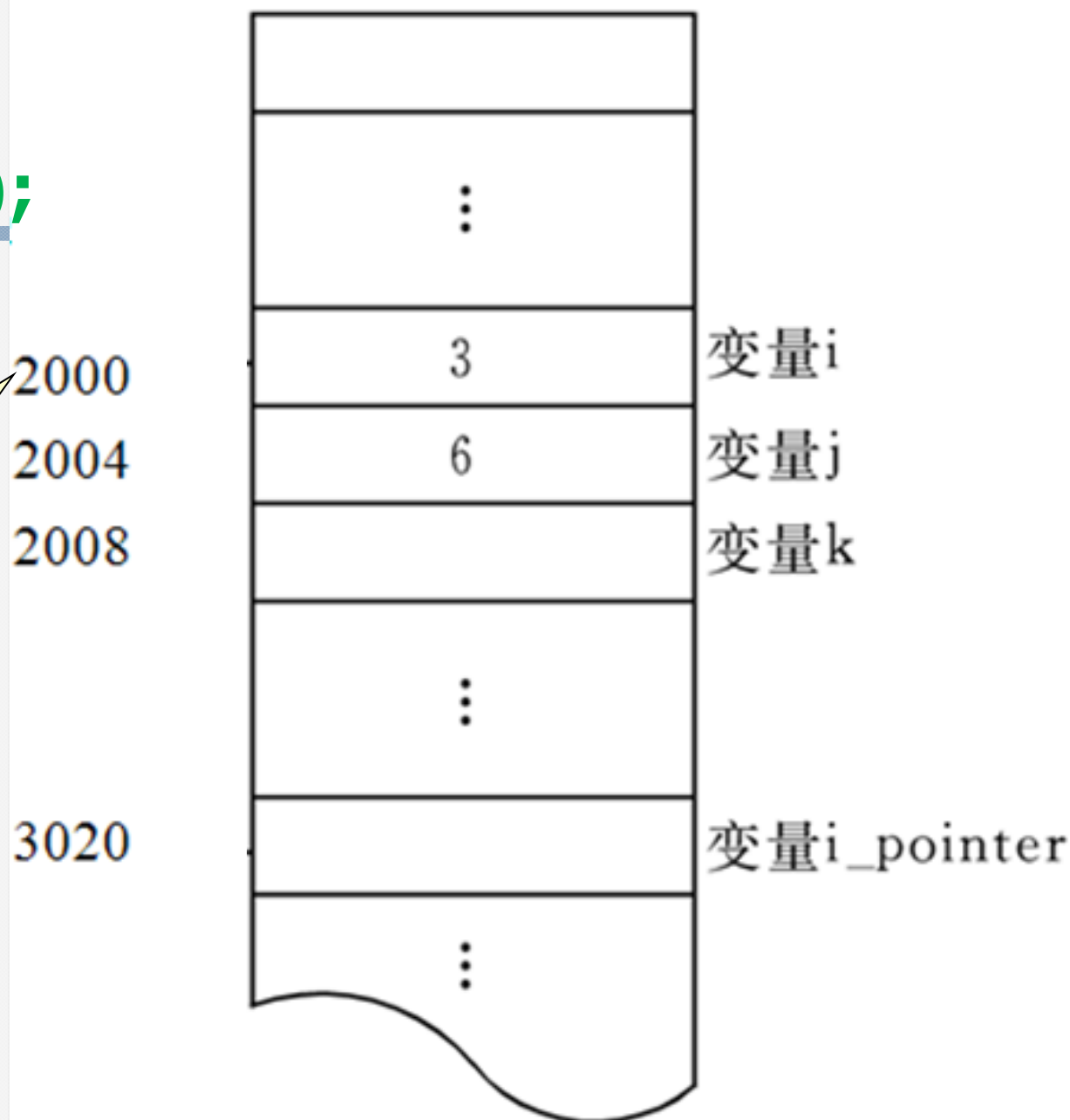
```
int i=3,j=6,k;
```

```
printf("%d",i);
```

通过变量名*i*

找到*i*的地址  
**2000**，从该存储  
单元读取到**3**

## 内存用户数据区





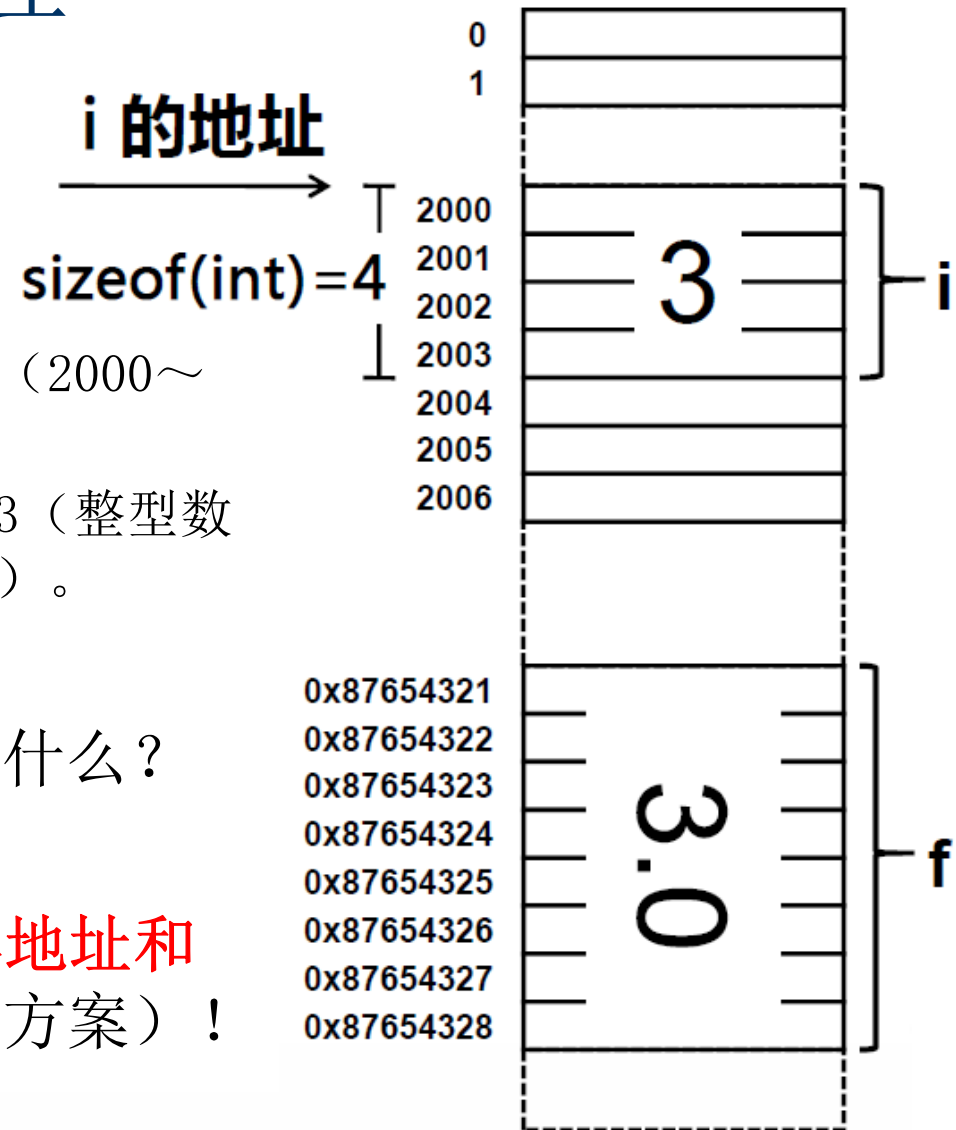
# 访问变量的过程

## ■ 执行“i=3;”会发生什么？

1. 找到i 的地址（2000）；
2. 确定i 所占的存储空间范围（2000～2003，4个字节）；
3. 往这存储空间写入整型数值3（整型数据通常采用32位二进制补码）。

## ■ 接下来，分析“f=i;”将发生什么？

- ## ■ 可见，访问变量必须知道其地址和类型（存储空间大小及编码方案）！



**int i=3,j=6,k;**

**k=i+j;**

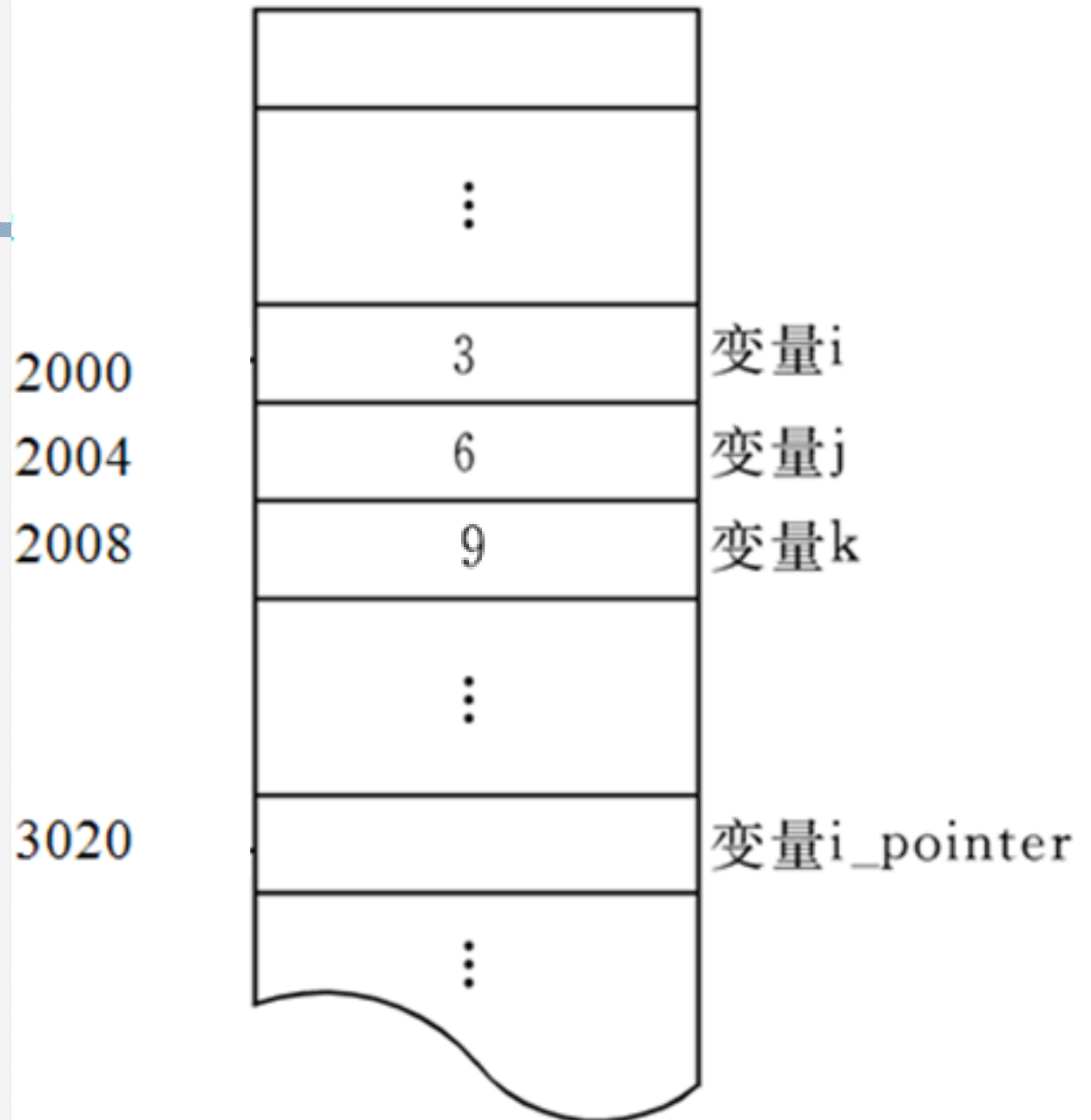
从这里取**3**

从这里取**6**

将**9**送到这里

直接存取

## 内存用户数据区



```
int i=3,j=6,k;
```

定义特殊变量 `i_pointer`

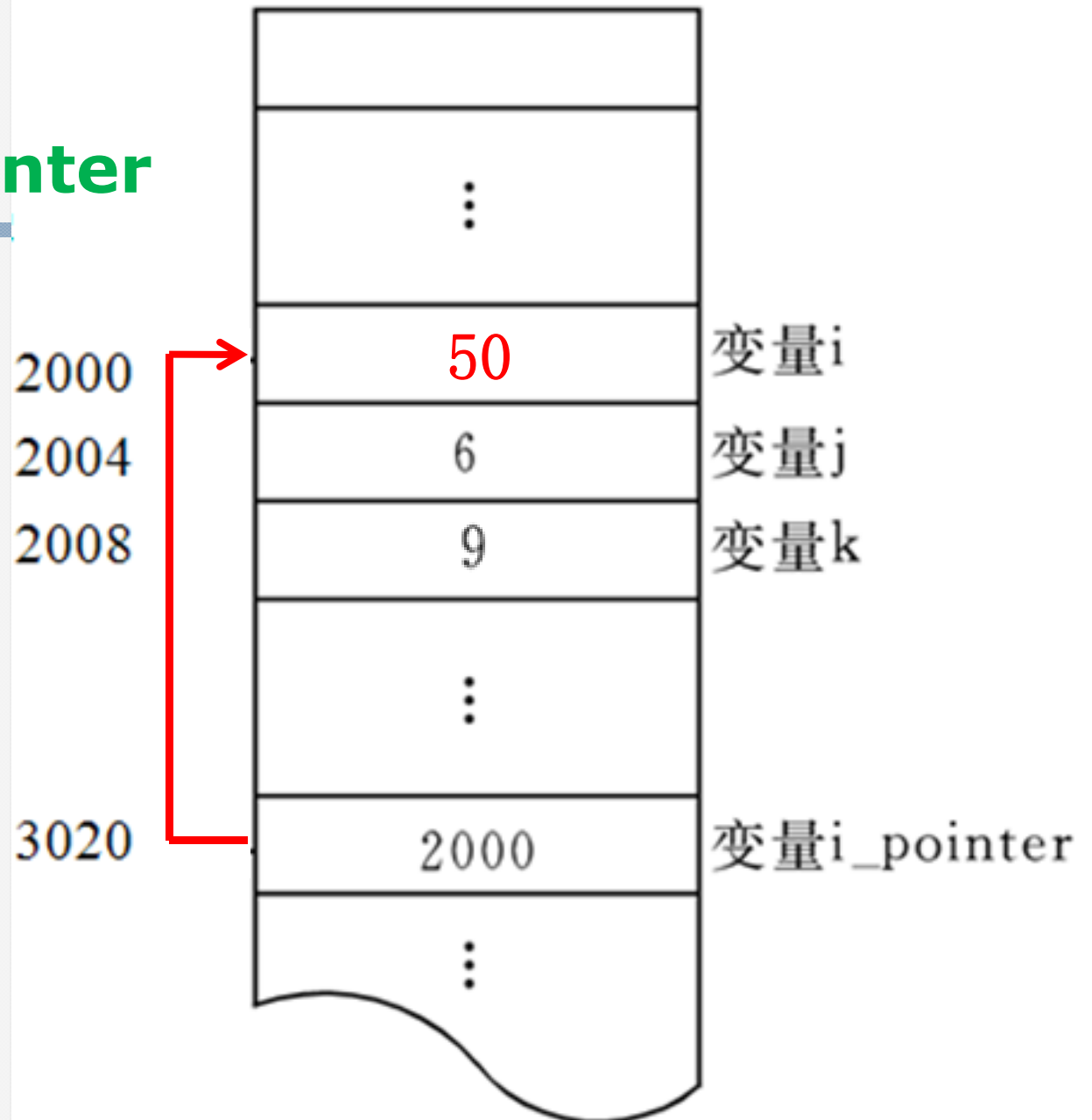
```
i_pointer=&i;
```

```
*i_pointer=50;
```

将 `i` 的地址  
存到这里

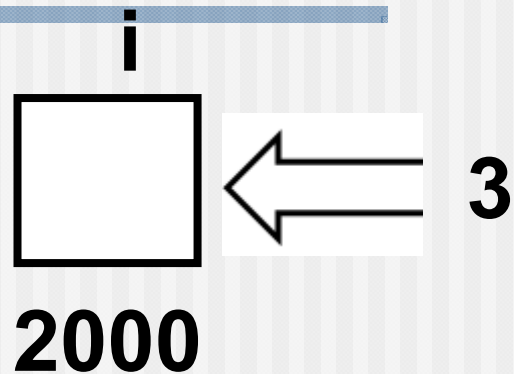
间接存取

内存用户数据区



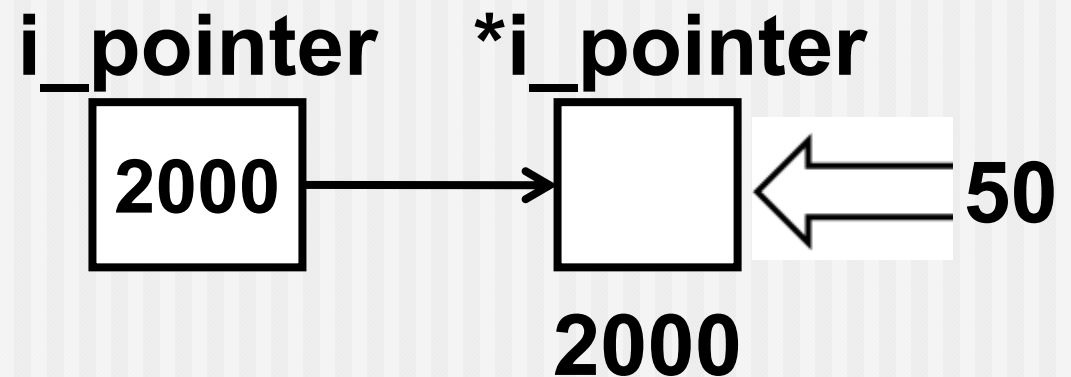
直接存取

$i=3$



间接存取

$*i\_pointer=50;$



## 8.1 指针是什么 (3)

### ■ 直接访问与间接访问

- 按变量地址直接存取变量值的方法称为“**直接访问**”。

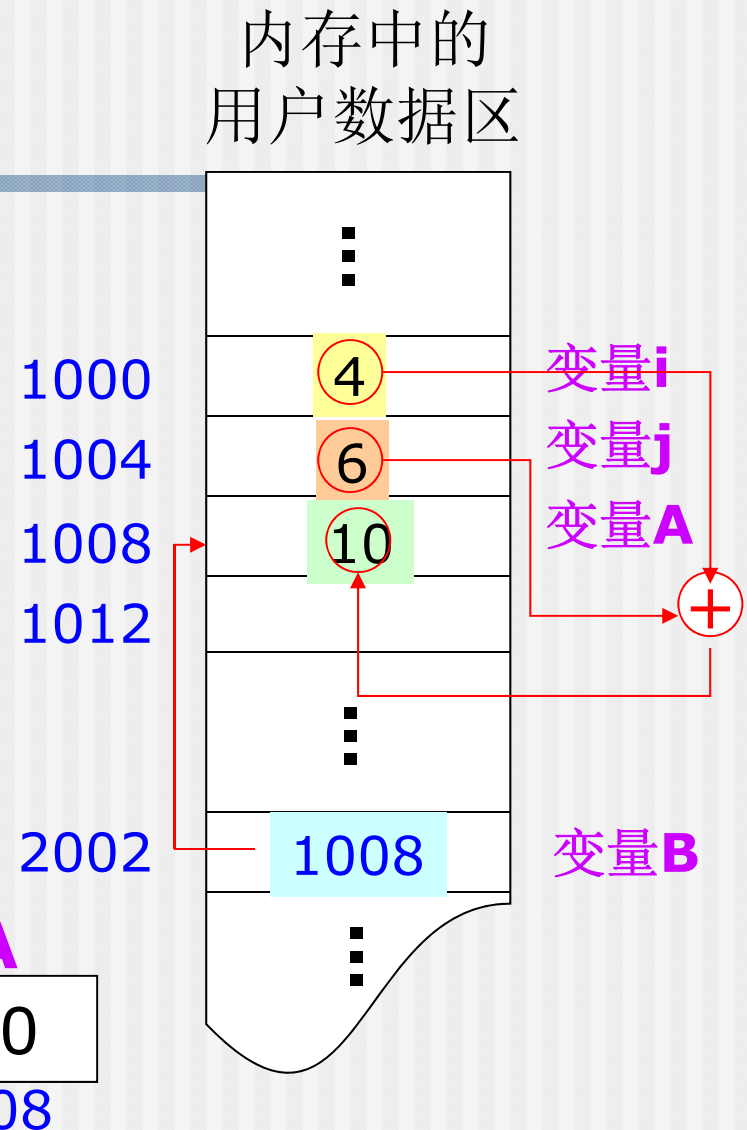
例:  $A = i + j$

- 将变量A的地址存放在另一个变量B中。要存取变量A的值时，先找到变量B，从中得到变量A的地址，然后到该地址取得变量A的值。这就是所谓的“**间接访问**”。

例:  $B = \&A;$   
 $*B = 3;$

“间接访问”运算符

取地址运算符



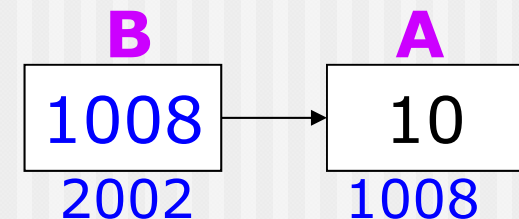
## 8.1 指针是什么（4）

### ■ 指针与指针变量

- 由于通过地址能够找到所需的内存单元（/变量），即地址“**指向**”该内存单元，因此在C语言中将地址形象化地成为“**指针**”，意为通过它能够找到以它为地址的内存单元。
- 一个变量的地址，称为该**变量的“指针”**
- 一个专门用来存放另一变量的地址（即指针）的变量，称为**指针变量**。

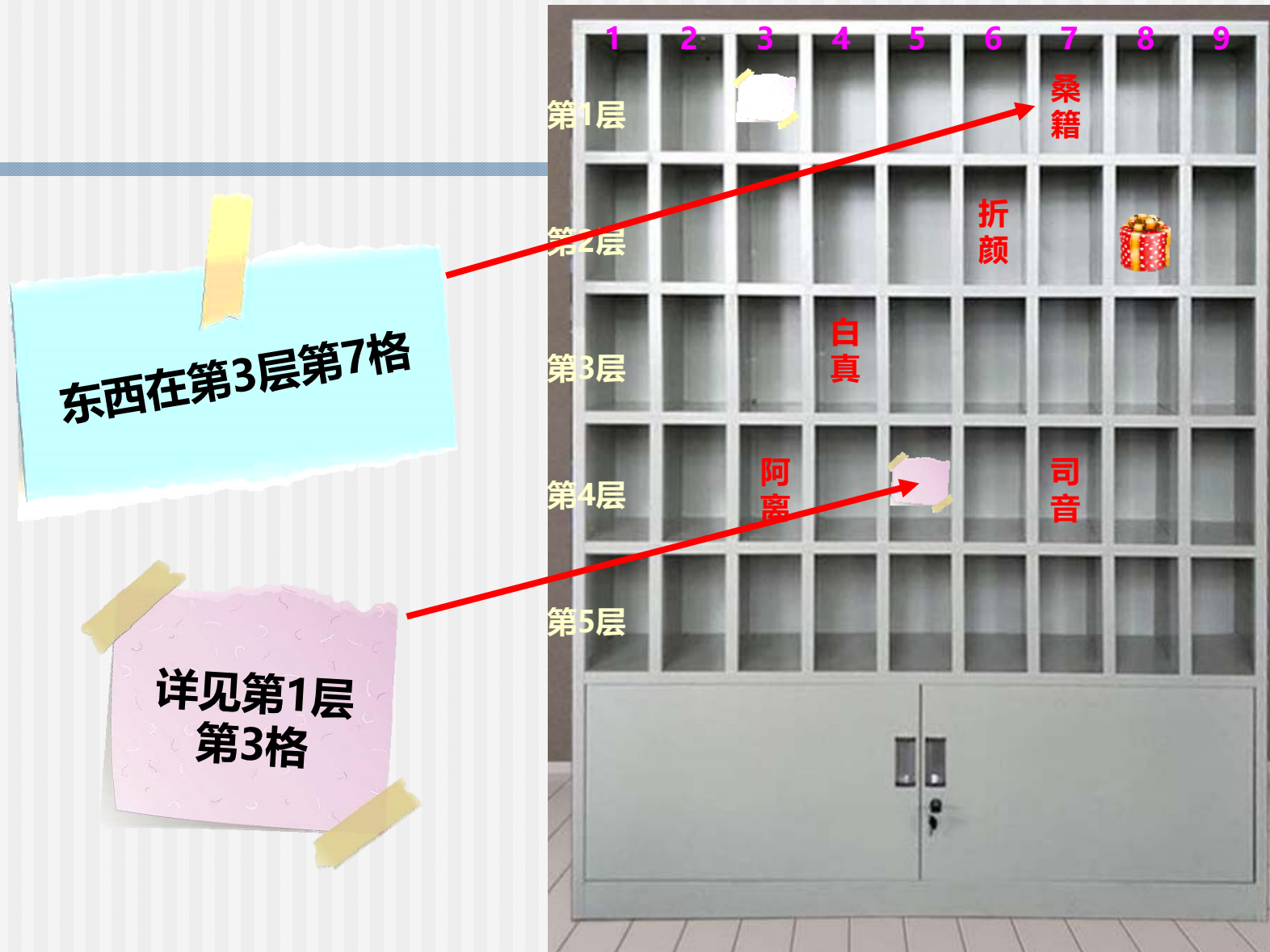
- 指针变量的值是地址（即指针）。

例：**B=&A;**



- 务必区分“指针”和“指针变量”这两个概念！
  - 指针变量是专门用来存放地址的变量！





## 8.2 指针变量

---

8.2.1 使用指针变量的例子

8.2.2 怎样定义指针变量

8.2.3 怎样引用指针变量

8.2.4 指针变量作为函数参数

## 8.2.1 使用指针变量的例子

---

**【例8.1】** 通过指针变量访问整型变量。

**【解题思路】** 先定义2个整型变量，再定义2个指针变量，分别指向这两个整型变量，通过访问指针变量，可以找到它们所指向的变量，从而得到这些变量的值。

```
a=100,b=10  
*pointer_1=100,*pointer_2=10
```

```
#include <stdio.h>
```

```
int main()
```

```
{ int a=100,b=10;
```

```
    int *pointer_1, *pointer_2;
```

```
    pointer_1=&a;
```

```
    pointer_2=&b;
```

```
    printf("a=%d,b=%d\n",a,b);
```

```
    printf("*pointer_1=%d,*pointer_2=  
           %d\n",*pointer_1,*pointer_2);
```

```
    return 0;
```

```
}
```

定义两个指针变量

直接访问，输出变量a和b的值

间接访问，输出变量a和b的值

```
#include <stdio.h>
int main()
```

此处\*与类型名在一起，  
共同定义指针变量

```
{ int a=100,b=10;
  int*pointer_1,*pointer_2;
  pointer_1=&a;
  pointer_2=&b;
  printf("a=%d,b=%d\n",a,b);
  printf("*pointer_1=%d,*pointer_2=
    %d\n",*pointer_1,*pointer_2);
  return 0;
}
```

此处\*与指针变量一起使用，代表  
指针变量所指向的内存单元。



## 8.2.2 怎样定义指针变量

■ 定义一个指针变量：指针类型

【一般形式】 基类型 \* 指针变量名;

例: `float *pointer_1, *pointer_2;`  
`int *pointer_3;`

■ 基类型

- 可以是任何有效
- 指定了该指针变

读作：“指向int的指针”  
或“int指针”

- 不同类型的数据在内存中所占的字节数和存放方式都是不同的，指针的移动和指针的运算（加、减）等都需要这方面的信息。

■ “\*” 既不属于基类型也不属于变量，它表示所定义的变量是指针类型的。

■ 严格地说，指针 ≠ 地址

- 一个变量的指针包含：（1）变量的存储单元的地址信息  
（2）变量的存储单元的数据类型信息。



- 下列定义实现了怎样的功能？

**int \* a, b, c;**

结果是：定义了一个指针变量a，两个整型变量b, c;

- 如果想将a,b,c都定义成指针变量，需要：

**int \*a,\*b,\*c;**

或 **int \*a; int \*b; int \*c;**

## 8.2.2 怎样定义指针变量

### ■ 指针变量的初始化和赋值

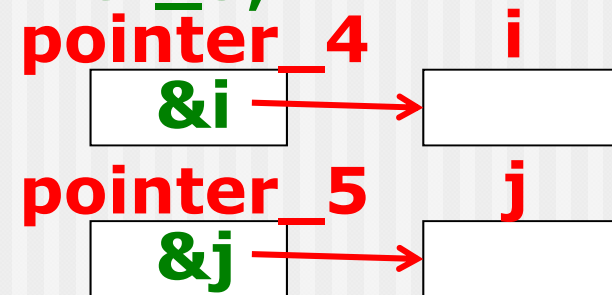
- 赋值语句：将一个变量的地址赋给一个指针变量

例： **int i,j;**

**int \*pointer\_4, \*pointer\_5;**

**pointer\_4=&i;**

**pointer\_5=&j;**



- 指针变量初始化

例： **int \*pointer\_6=&i;**

**【注意】** 通常，只有数据类型与指针变量的基类型一致的变量的地址才能放到该指针变量中。

错例： **float \*pointer\_7=&j;** 编译时会有警告！

**【注意】** 指针变量中只能存放地址（指针），不要将整型量/其它任何非地址类型的数据赋给指针变量！

错例： **pointer\_7=1000;**

- 在声明一个指针变量后，若没有经过初始化，指针变量的值是不确定的，直接使用未加初始化的指针变量可能会给程序带来各种内存错误。因此指针变量的初始化很重要。

- 如果一开始不知该将此指针指向何处，最简单的方式是利用**NULL**（符号常量，值为0）将指针置0，

如：**int \*pInt = NULL;**

- 类型指针变量只能存放该类型变量的地址

**int a, b;**

**int \*p1 = &a, \*p2 = &b;**

**float \*p3;**

**p3**  **&a;** 错误，**float**类型指针不能指向整型变量

## 8.2.3 怎样引用指针变量

- 在引用指针变量时，可能有二种情况：

- 给指针变量赋值。如：  
`p=&a;`

使p指向a

- 引用指针变量所指向的变量。如有

`p=&a; *p=1;`

则执行`printf("%d",*p);` 将输出1

- 引用指针变量的值。如：

`printf("%o",p);`

以八进制输出a的地址

## 示例：显示指针变量所占空间大小

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char * pChar;
```

```
    int * pInt;
```

```
    double * pDouble;
```

```
    printf("pChar size=%d\n", sizeof(pChar));
```

```
    printf("pInt size=%d\n", sizeof(pInt));
```

```
    printf("pDouble size=%d\n", sizeof(pDouble));
```

```
    return 0;
```

```
}
```

```
pChar size=4
pInt size=4
pDouble size=4
```

返回括号内的变量占的字节数



## 8.2.3 怎样引用指针变量（2）

### ■ 要熟练掌握两个有关的运算符：

(1) **&** 取地址运算符，取得存储单元的地址

**&a**是变量**a**的地址

(2) **\*** 指针运算符（“间接访问”运算符），读取变量所指向的存储单元的内容。

如果**p=&a**，即**p**指向变量**a**，则**\*p**就代表**a**。

**k=\*p;**            (把**a**的值赋给**k**)

**\*p=1;**            (把**1**赋给**a**)



**【例8.2】** 输入a和b两个整数，按先大后小的顺序输出a和b。

---

- 解题思路：用指针方法来处理这个问题。不交换整型变量的值，而是交换两个指针变量的值。

```
#include <stdio.h>
```

```
int main()
```

```
{ int *p1,*p2,*p,a,b;
```

```
    printf("integer numbers:");
```

```
    scanf("%d,%d",&a,&b);
```

5,9

```
    p1=&a; p2=&b;
```

```
    if(a<b) 成立
```

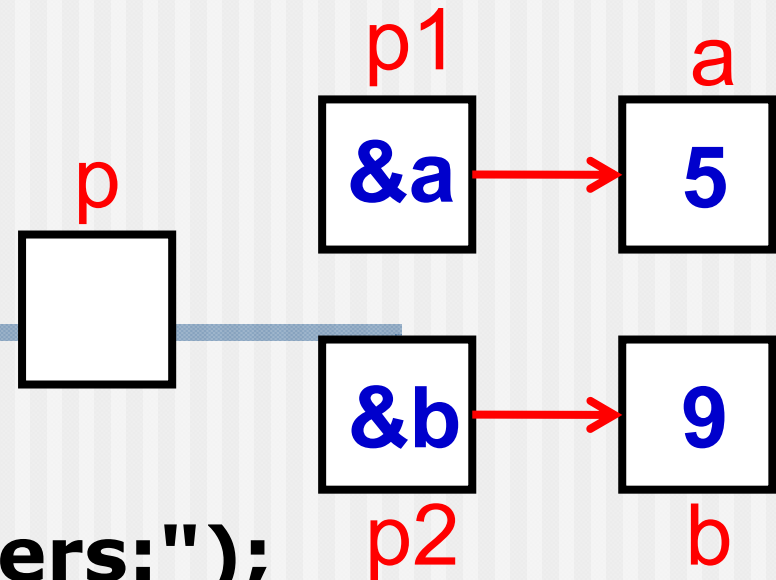
```
    { p=p1; p1=p2; p2=p; }
```

```
    printf("a=%d,b=%d\n",a,b);
```

```
    printf("%d,%d\n",*p1,*p2);
```

```
    return 0;
```

```
}
```



```
#include <stdio.h>
```

```
int main()
```

```
{ int *p1,*p2,*p,a,b;
```

```
    printf("integer numbers:");
```

```
    scanf("%d,%d",&a,&b);
```

```
    p1=&a;    p2=&b;
```

```
    if(a<b)
```

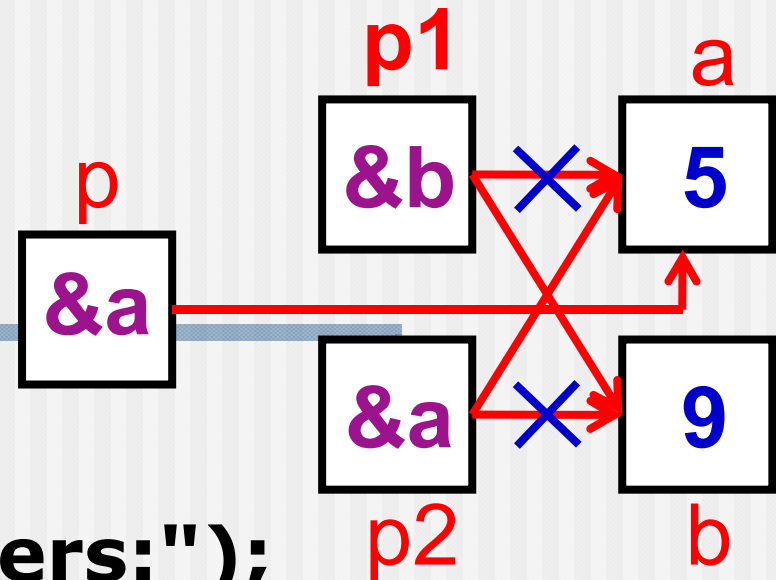
```
    { p=p1; p1=p2; p2=p; }
```

```
    printf("a=%d,b=%d\n",a,b);
```

```
    printf("%d,%d\n",*p1,*p2);
```

```
    return 0;
```

```
}
```



```
#include <stdio.h>
```

```
int main()
```

```
{ int *p1,*p2,*p,a,b;
```

```
printf("integer numbers:");
```

```
scanf("%d,%d",&a,&b);
```

```
p1=&a; p2=&b;
```

```
if(a<b)
```

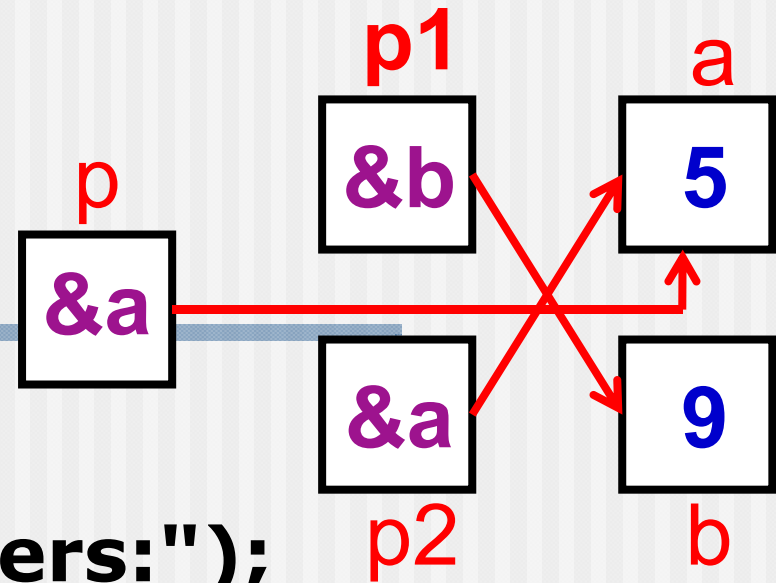
```
{ p=p1; p1=p2; p2=p; }
```

```
printf("a=%d,b=%d\n",a,b);
```

```
printf("%d,%d\n",*p1,*p2);
```

```
return 0;
```

```
}
```



```
a=5 , b=9  
9 , 5
```

## ■ 注意：

- a和b的值并未交换，它们仍保持原值
- 但p1和p2的值改变了。p1的值原为&a，后来变成&b，p2原值为&b，后来变成&a
- 这样在输出\*p1和\*p2时，实际上是输出变量b和a的值，所以先输出9，然后输出5
  - 通过交换两个指针变量的值来实现！

```
#include <stdio.h>
```

```
int main()
```

```
{ int *p1,*p2,*p,a,b;
```

```
printf("integer numbers:");
```

```
scanf("%d,%d",&a,&b);
```

```
p1=&a; p2=&b;
```

```
if(a<b)
```

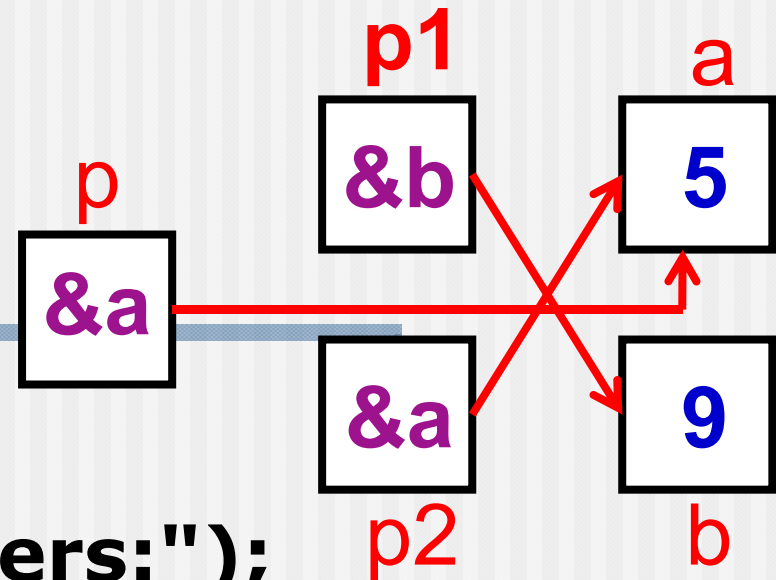
```
{ p=p1; p1=p2; p2=p; }
```

```
printf("a=%d,b=%d\n",a,b);
```

```
printf("%d,%d\n",*p1,*p2);
```

```
return 0;
```

```
}
```



可否改为  $p1 = \&b;$   $p2 = \&a;$  ?

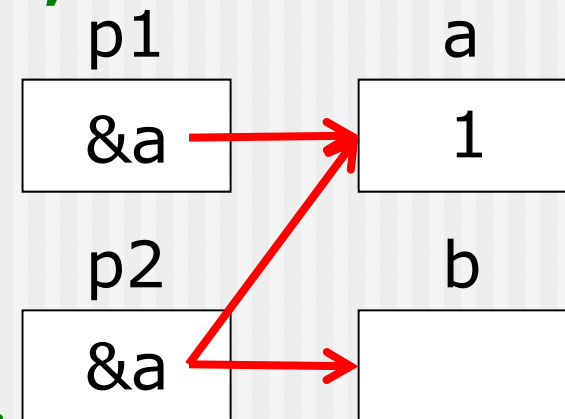


## 8.2.3 怎样引用指针变量（3）

### ■ 指针运算符

- **&**和**\***均为单目运算符，优先级相同，自右向左结合；
- **&**和**\*** 是互逆运算

例： **int a,b, \*p1=&a, \*p2=&b;**  
**p2=&\*p1;**  
**\*&a=1; //等价于\*p1=1**  
 即 **&\*p==p, \*&a==a。**



例：设**p1**为指针变量，且**p1=&a**，  
 则**(\*p1)++**等价于**a++**，  
 不同于**\*p1++**（即**\*(p1++)**）。

## 8.2.4 指针变量作为函数参数

【例8.3】 题目要求同例8.2，即对输入的两个整数按大小顺序输出。现用函数处理，而且用指针类型的数据作函数参数。

- 解题思路：定义一个函数**swap**，将指向两个整型变量的指针变量作为实参传递给**swap**函数的形参指针变量，在函数中通过指针实现交换两个变量的值。

```
#include <stdio.h>
```

```
int main()
```

```
{ void swap(int *p1,int *p2);
```

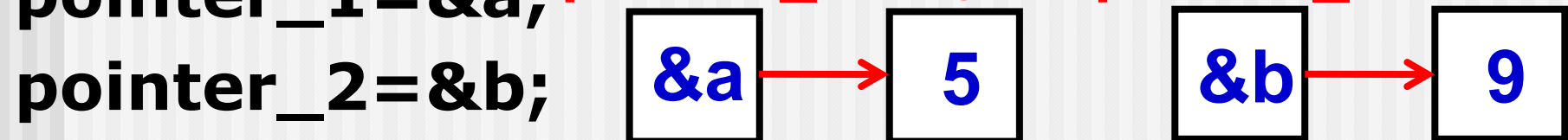
```
  int a,b; int*pointer_1,*pointer_2;
```

```
  printf("please enter a and b:");
```

```
  scanf("%d,%d",&a,&b);
```

5,9

```
  pointer_1=&a; pointer_1 a pointer_2 b
```



```
  if (a<b) swap(pointer_1,pointer_2);
```

```
  printf("max=%d,min=%d\n",a,b);
```

```
  return 0;
```

```
}
```

```
void swap(int *p1,int *p2)
```

```
{ int temp;
```

```
temp=*p1;
```

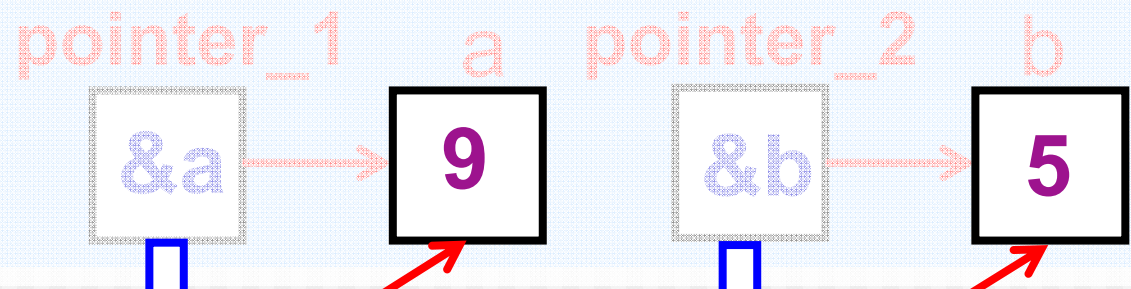
```
*p1=*p2;
```

```
*p2=temp;
```

```
}
```

main

```
please enter a and b:5,9  
max=9,min=5
```



```
void swap(int *p1,int *p2)
{ int temp;
  temp=*p1;
  *p1=*p2;
  *p2=temp;
}
```

错!!!  
temp指针未赋值!

【注意】引用尚未初始化的指针变量是十分危险的!

```
void swap(int *p1,int *p2)
{ int *temp;
  *temp=*p1;
  *p1=*p2;
  *p2=*temp;
}
```

```
#include <stdio.h>
```

```
int main()
```

```
{ void swap(int *p1,int *p2);
```

```
int a,b; int*pointer_1,*pointer_2;
```

```
printf("please enter a and b:");
```

```
scanf("%d,%d",&a,&b);
```

```
pointer_1=&a;
```

```
pointer_2=&b;
```

```
if (a<b) swap( &a, &b );
```

```
printf("max=%d,min=%d\n",a,b);
```

```
return 0;
```

```
}
```



```
#include <stdio.h>
```

```
int main()
```

```
{.....
```

```
    if (a<b) swap(a,b);
```

```
    printf("max=%d,min=%d\n",a,b);
```

```
    return 0;
```

```
}
```

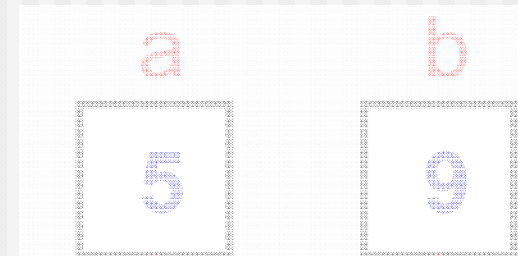
```
void swap(int x,int y)
```

```
{ int temp;
```

```
    temp=x; x=y; y=temp;
```

```
}
```

错!!!  
无法交换a,b



**【注意】**

- 1) 实参变量和形参变量之间的数据传递是单向的“值传递”！
- 2) 可修改是实参所指向的变量的值，而不是实参本身的值！

■ 如果想通过函数调用得到  $n$  个要改变的变量：

- ① 在主调函数中设  $n$  个变量，用  $n$  个指针变量指向它们
- ② 设计一个函数，有  $n$  个指针形参。在这个函数中**改变这  $n$  个形参所指向的  $n$  个变量的值**
- ③ 在主调函数中调用这个函数，在调用时将这  $n$  个指针变量作实参，将它们的地址传给该函数的形参
- ④ 在执行该函数的过程中，通过形参指针变量，改变它们所指向的  $n$  个变量的值
- ⑤ 主调函数中就可以使用这些改变了值的变量

## 【例8.4】对输入的两个整数按大小顺序输出。

- 解题思路：尝试调用**swap**函数来实现题目要求。在函数中改变形参(指针变量)的值，希望能由此改变实参(指针变量)的值。

```
#include <stdio.h>
```

```
int main()
```

```
{ void swap(int *p1, int *p2);
```

```
int a, b; int *pointer_1, *pointer_2;
```

```
scanf("%d, %d", &a, &b);
```

```
pointer_1 = &a; pointer_2 = &b;
```

```
if (a < b) swap(pointer_1, pointer_2);
```

```
printf("max=%d, min=%d", a, b);
```

```
return 0; max=5, min=9
```

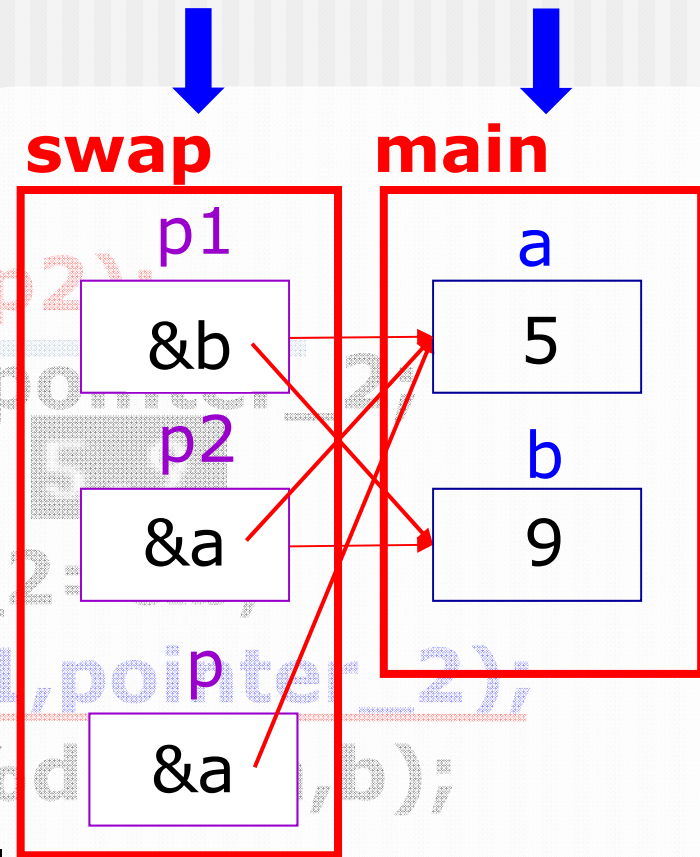
```
}
```

```
void swap(int *p1, int *p2)
```

```
{ int *p;
```

```
    p = p1; p1 = p2; p2 = p;
```

```
}
```



错!!!  
只交换形参指向

---

**【注意】** 函数的调用可以（而且只可以）得到一个返回值（即函数值），而使用指针变量作参数，可以得到多个变化了的值。如果不用指针变量是难以做到这一点的。

- 要善于利用指针法。
  - 能完成一些直接访问无法实现的功能
  - 突破屏障去操作计算机底层，提供更大的编程灵活度



**【例8.5】** 输入3个整数a,b,c，要求按由大到小的顺序将它们输出。用函数实现。

---

- 解题思路：采用例8.3的方法在函数中改变这3个变量的值。用swap函数交换两个变量的值，用exchange函数改变这3个变量的值。



```
#include <stdio.h>
```

```
int main()
```

```
{ void exchange(int *q1, int *q2, int *q3);
```

```
int a,b,c,*p1,*p2,*p3;
```

```
scanf("%d,%d,%d",&a,&b,&c);
```

20,-54,67

```
p1=&a;p2=&b;p3=&c;
```

```
exchange(p1,p2,p3);
```

调用结束后不会  
改变指针的指向

```
printf("%d,%d,%d\n",a,b,c);
```

```
return 0;
```

```
}
```

```
void exchange(int *q1, int *q2, int *q3)
```

```
{ void swap(int *pt1, int *pt2);
```

```
    if(*q1<*q2) swap(q1,q2);
```

```
    if(*q1<*q3) swap(q1,q3);
```

```
    if(*q2<*q3) swap(q2,q3);
```

```
}
```

```
void swap(int *pt1, int *pt2)
```

```
{ int temp;
```

```
    temp=*pt1; *pt1=*pt2; *pt2=temp;
```

```
}
```

交换指针指向的变量值

20,-54,67  
67,20,-54



## 8.3 通过指针引用数组

---

8.3.1 数组元素的指针

8.3.2 在引用数组元素时指针的运算

8.3.3 通过指针引用数组元素

8.3.4 用数组名作函数参数

8.3.5\* 通过指针引用多维数组

## 8.3.1 数组元素的指针

- 一个变量有地址，一个数组包含若干元素，每个数组元素都有相应的地址
- 指针变量可以指向数组元素（把某一元素的地址放到一个指针变量中）
- 所谓**数组元素的指针**就是数组元素的地址（及其数据类型）



- 可以用一个指针变量指向一个数组元素

```
int a[10]={1,3,5,7,9,11,13,15,17,19};
```

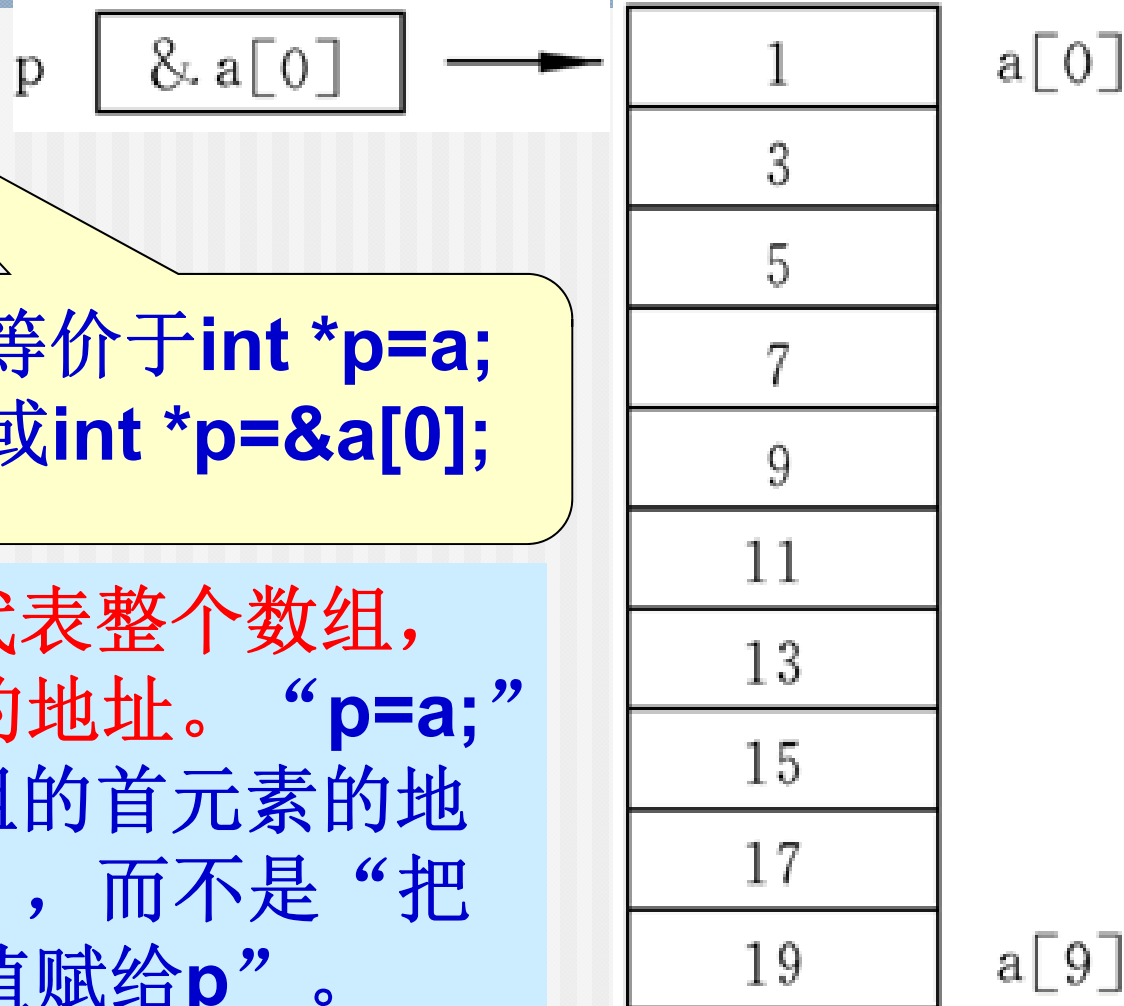
```
int *p;
```

```
p=&a[0];
```

等价于 **p=a;**

等价于 **int \*p=a;**  
或 **int \*p=&a[0];**

注意：数组名**a**不代表整个数组，只代表数组首元素的地址。“**p=a;**”的作用是“把**a**数组的首元素的地址赋给指针变量**p**”，而不是“把数组**a**各元素的值赋给**p**”。



## 示例：数组与指针

```
#include <stdio.h>
int main()
{
    int a[10]={0,1,2,3,4,5,6,7,8,9};
    int *p;
    p = &a[0];

    printf("a[0]=%X;\n", a[0]);
    printf("&a[0]=%X;\n", &a[0]);
    printf("&a=%X;\n", &a);
    printf("a=%X;\n", a);
    printf("p=%X;\n", p);
    printf("*p=%X;\n", *p);

    return 0;
}
```

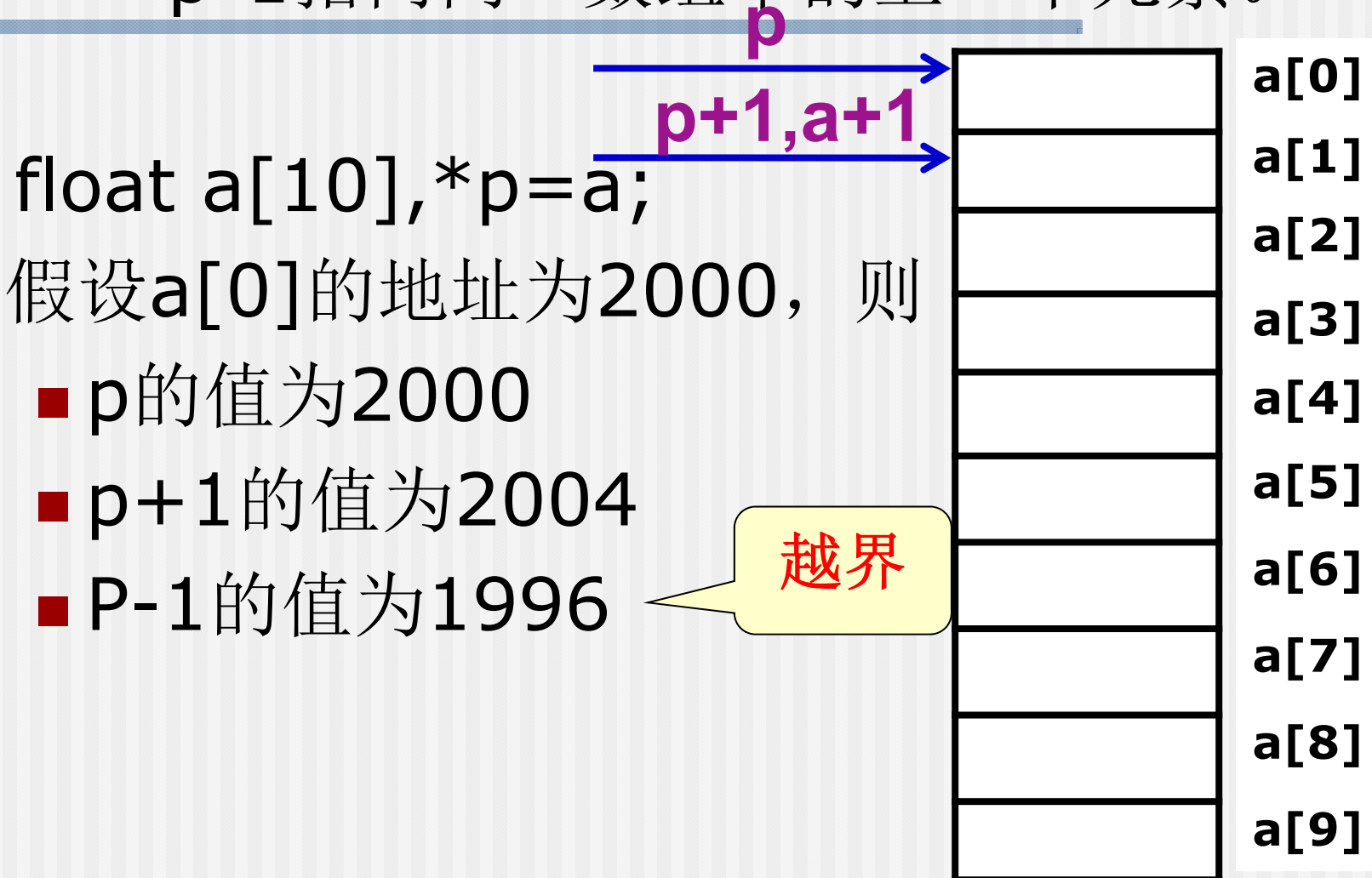
```
a[0]=0;
&a[0]=22FF14;
&a=22FF14;
a=22FF14;
p=22FF14;
*p=0;
```



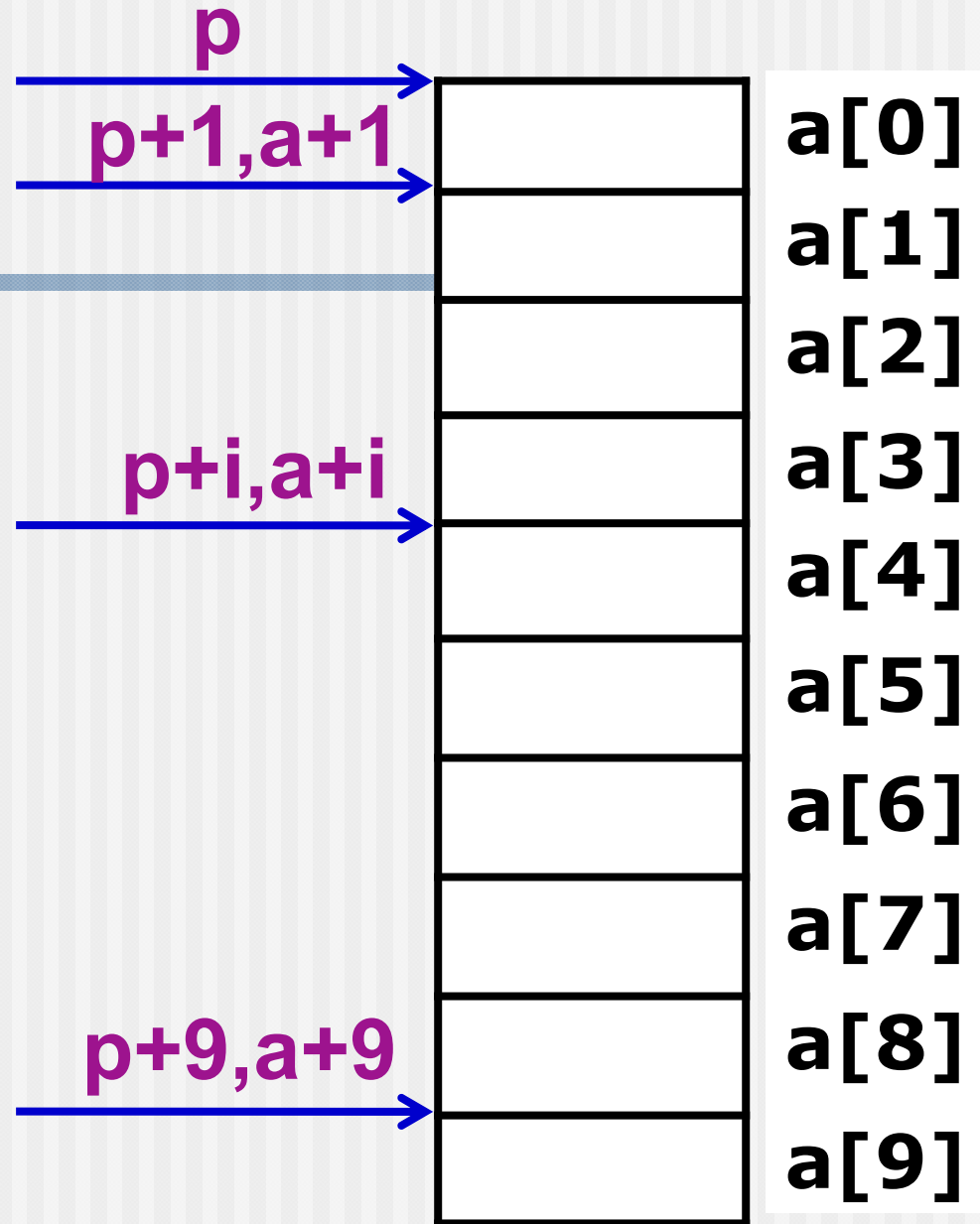
## 8.3.2 在引用数组元素时指针的运算

- 数组元素在内存中是顺序地、连续地存储的，即逻辑上相邻的数组元素在物理地址上也是相邻的，因此，我们先将指针 $p$ 指向数组 $a$ 的第一个元素，继而就可以通过将指针 $p$ 向前或者向后移动来访问数组 $a$ 中的任意元素。
- 在指针指向数组元素时，允许以下运算：
  - 加一个整数(用 $+$ 或 $+=$ )，如 $p+1$
  - 减一个整数(用 $-$ 或 $-=$ )，如 $p-1$
  - 自加运算，如 $p++$ ， $++p$
  - 自减运算，如 $p--$ ， $--p$
  - 两个指针相减，如 $p1-p2$  (只有 $p1$ 和 $p2$ 都指向同一数组中的元素时才有意义)

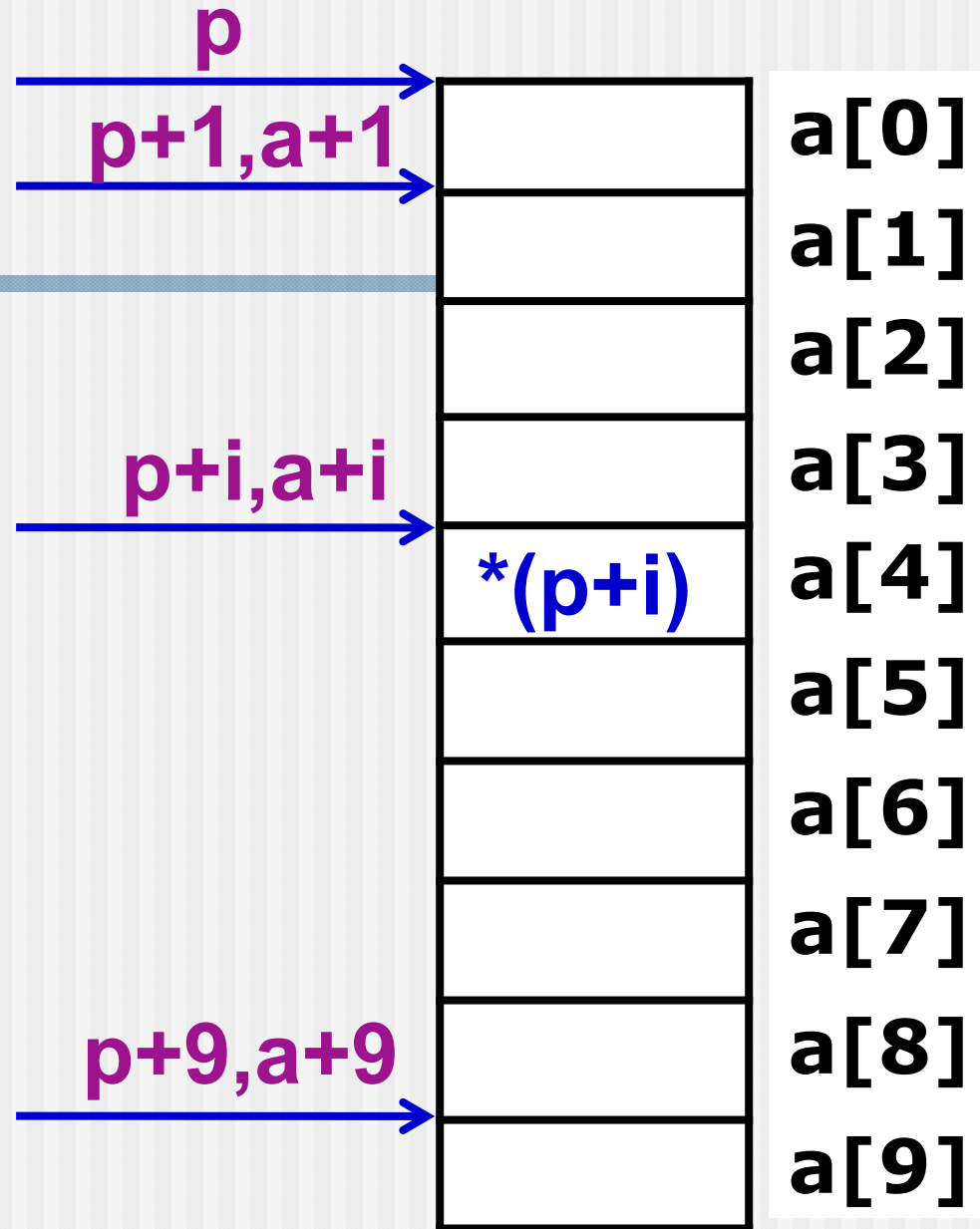
(1) 如果指针变量 $p$ 已指向数组中的一个元素，  
则： $p+1$ 指向同一数组中的下一个元素，  
 $p-1$ 指向同一数组中的上一个元素。



(2) 如果  $p$  的初值为  $\&a[0]$ , 则  $p+i$  和  $a+i$  就是数组元素  $a[i]$  的地址, 或者说, 它们指向  $a$  数组序号为  $i$  的元素



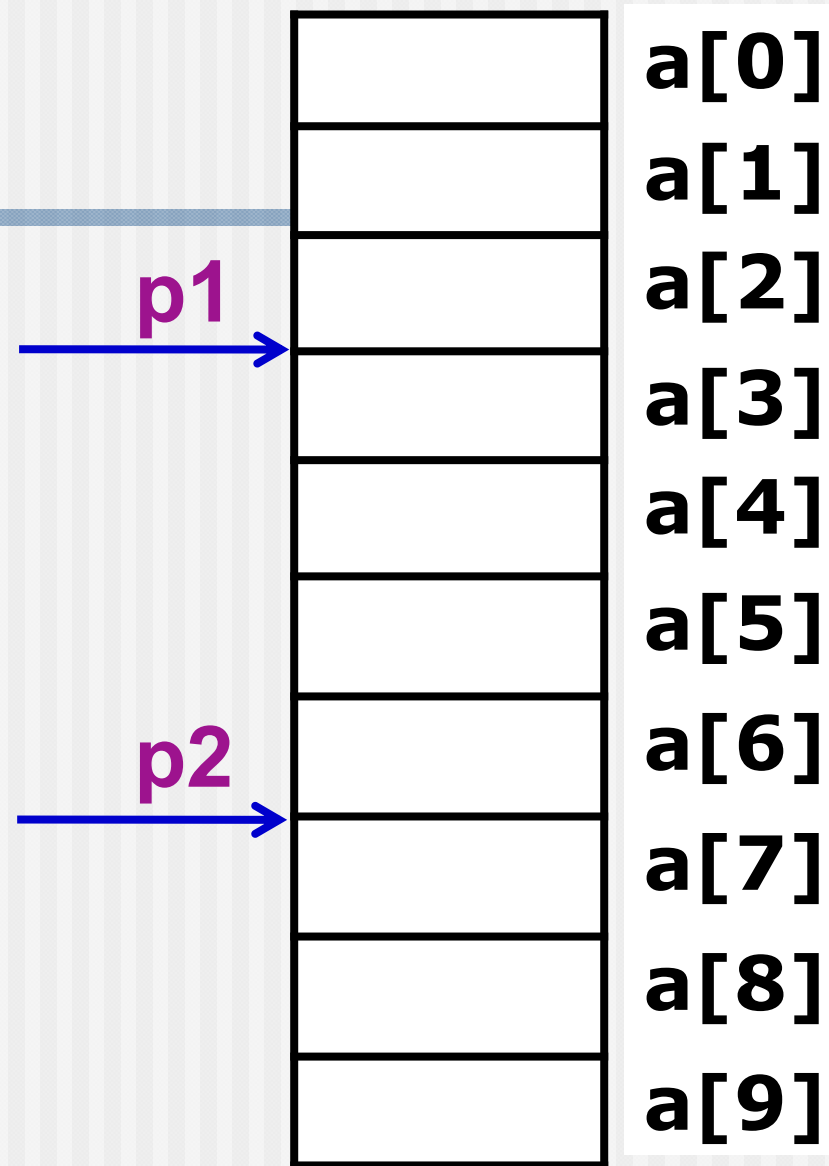
(3)  $*(p+i)$ 或  
 $*(a+i)$ 是 $p+i$   
或 $a+i$ 所指向的  
数组元素，即  
 $a[i]$ 。



(4) 如果指针p1和p2  
都指向同一数组

p2-p1的值是4

【注意】不能计算  
p1+p2，因为没有  
任何意义！

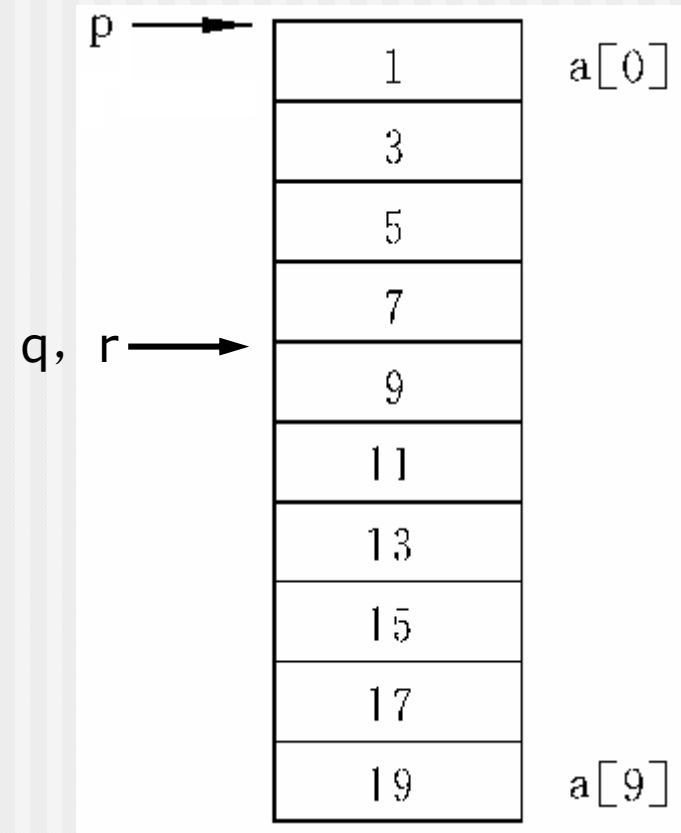


## 8.3.2 在引用数组元素时指针的运算

### ■ 通过指针引用数组元素

#### ■ 指针的运算：

- 指针与整数做加减法运算  
如： **$q-2$ ,  $r+3$ ,  $p++$**
- 指向同一数组的指针之间的相减及比较运算  
如： **$q==r$ ,  $q-p>0$ ,  $q>p$**
- 除上述情况外，指针没有其它有意义的算术运算或位运算！  
错例如： **$q+r$ ,  $\&p>\&q$**





### 8.3.3 通过指针引用数组元素

- 引用一个数组元素，可用下面两种方

(1) **下标法**，如 $a[i]$ 形式

(2) **指针法**，如 $*(a+i)$ 或 $*(p+i)$

其中 $a$ 是数组名， $p$ 是指向数组元素的指针变量，其初值 $p=a$

- $[]$ 实质上是**变址运算符**，指针变量 $[i]$ 即“指针变量 $+i$ ”
- 在C语言中，基类型相同的指针变量和数组有一定的通用性。

**【例8.6】** 有一个整型数组**a**，有**10**个元素，要求输出数组中的全部元素。

---

- 解题思路：引用数组中各元素的值有**3**种方法：**(1)**下标法；**(2)**通过数组名计算数组元素地址，找出元素的值；**(3)**用指针变量指向数组元素
- 分别写出程序，依此比较分析。

(1) 下标法。

```
#include <stdio.h>
```

```
int main()
```

```
{ int a[10]; int i;
```

```
    printf("enter 10 integer numbers:\n");
```

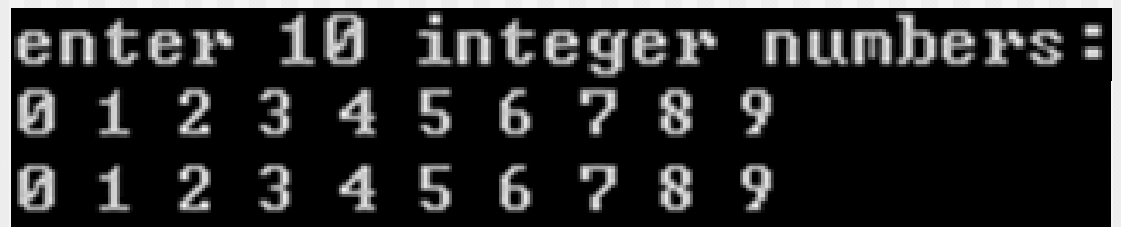
```
    for(i=0;i<10;i++) scanf("%d",&a[i]);
```

```
    for(i=0;i<10;i++) printf("%d ",a[i]);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

A screenshot of a terminal window showing the output of the C program. The first line is the prompt "enter 10 integer numbers:". The second line shows the user input "0 1 2 3 4 5 6 7 8 9". The third line shows the program's output "0 1 2 3 4 5 6 7 8 9".

```
enter 10 integer numbers:  
0 1 2 3 4 5 6 7 8 9  
0 1 2 3 4 5 6 7 8 9
```

(2) 通过数组名计算数组元素地址，找出元素的值

**#include <stdio.h>**

**int main()**

**{ int a[10]; int i;**

**printf("enter 10 integer numbers:\n");**

**for(i=0;i<10;i++) scanf("%d",&a[i]);**

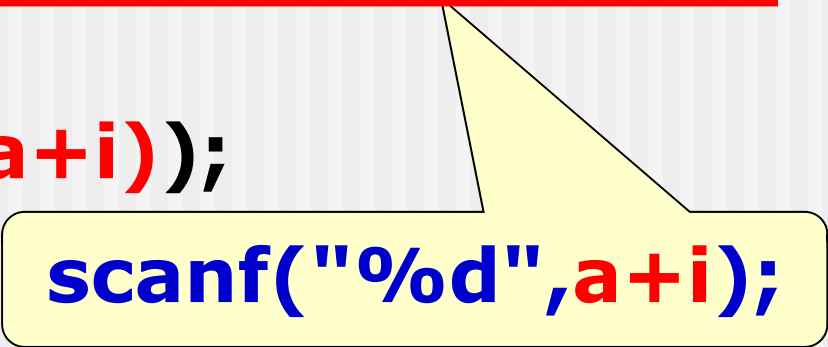
**for(i=0;i<10;i++)**

**printf("%d ",\*(a+i));**

**printf("\n");**

**return 0;**

**}**



**scanf("%d",a+i);**

### (3) 用指针变量指向数组元素

```
#include <stdio.h>
```

```
int main()
```

```
{ int a[10]; int
```

```
for(p=a;p<(a+10);p++)  
    scanf("%d",p);
```

```
    printf("enter 10 integer number:\n");
```

```
    for(i=0;i<10;i++) scanf("%d",&a[i]);
```

```
    for(p=a;p<(a+10);p++)  
        printf("%d ",*p);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

```
for(p=a;p<(a+10);a++)  
    printf("%d ",*a); 错!
```

## ■ 3种方法的比较:

### ① 第(1)和第(2)种方法**执行效率**相同

- C编译系统是将 $a[i]$ 转换为 $*(a+i)$ 处理的,即先计算元素地址。
- 因此用第(1)和第(2)种方法找数组元素费时较多。

### ② 第(3)种方法比第(1)、第(2)种方法快

- 用指针变量直接指向元素,不必每次都重新计算地址,像 $p++$ 这样的自加操作是比较快的
- 这种有规律地改变地址值( $p++$ )能大大提高执行效率

### ③ 用下标法比较**直观**,能直接知道是第几个元素。

- 用地址法或指针变量的方法不直观,难以很快地判断出当前处理的是哪一个元素。



**【例8.7】** 通过指针变量输出整型数组a的10个元素。

---

■ 解题思路：

用指针变量p指向数组元素，通过改变指针变量的值，使p先后指向a[0]到a[9]各元素。

```
#include <stdio.h>
```

```
int main()
```

```
{ int *p,i,a[10];
```

```
    p=a;
```

```
    printf("enter 10 integer numbers:\n");
```

```
    for(i=0;i<10;i++) scanf("%d",p++);
```

```
    p=a;
```

```
    for(i=0;i<10;i++,p++)
```

```
        printf("%d ",*p);
```

```
    printf("\n");
```

```
    return 0;
```

```
}
```

重新执行  
**p=a;**

指针变量可以指向数组之后的内存单元，编译程序并不认为是非法的，但应避免出现这样的情况，这会使程序得不到预期的结果。

退出循环时**p**指向**a[9]**后面的存储单元

因此执行此循环出问题

### 8.3.3 通过指针引用数组元素（2）

- 指针使用中应注意的问题：

- 可修改指针变量的值（如`p++`），使其指向不同的数据元素；**数组名虽然也是地址或指针，但应将其看成是指针常量**，`a++`是错误的。

- 未赋值的指针变量，其值不可预见。使用未赋值的指针变量将干扰程序甚至操作系统的正常运行！

错例如：**`int *p; *p=1;`**

## 8.3.3 通过指针引用数组元素（3）

### ■ 指针使用中应注意的问题：

#### ■ 注意指针变量的运算

例如：p指向数组a的首元素（即 $p=a$ ）

#### ➤ $p++$ （或 $p+=1$ ）

使p指向下一元素，即 $a[1]$ ；\*p得到 $a[1]$ 的值；

#### ➤ $*p++$

$++$ 与 $*$ 同级且均为右结合，等价于 $*(p++)$ ；

#### ➤ $(*p)++$

使p所指向的元素值加1，而不是p的指针值加1；

若 $p=a$ ，则 $(*p)++$ 相当于 $(a[0])++$ ；

#### ➤ $*(p++)$ 与 $*(++p)$

前者先取\*p的值，然后使p加1；后者先使p加1再取\*p

#### ➤ $*(p--)$ 与 $*(--p)$

## 8.3.4 用数组名作函数参数

- 用数组名作函数参数时，因为实参数组名代表该数组首元素的地址，形参应该是一个指针变量

**【表象】** 为什么函数的形参定义看起来是一个数组？

**【真相】** 当函数形参是一个数组名的时候，C编译器都是将形参数组名作为指针变量来处理的！

```
int main()
```

```
{ void fun(int arr[],int n);
```

```
  int array[10];    :
```

```
  :
```

```
  fun (array,10);
```

```
  return 0;
```

```
}
```

**fun(int \*arr,int n)**

```
void fun(int arr[ ],int n)
```

```
{ : }
```



```
int main()
```

```
{ void fun(int arr[],int n);
```

```
  int array[10];
```

```
  |
```

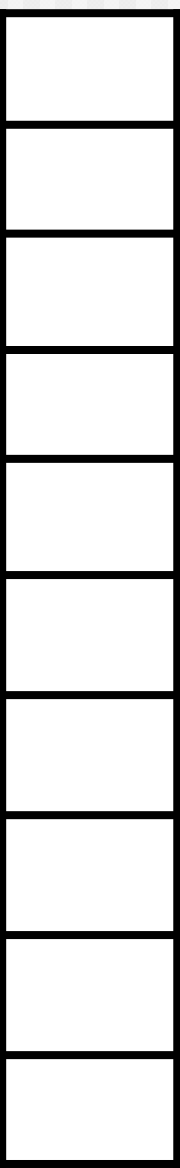
```
  fun (array,10);
```

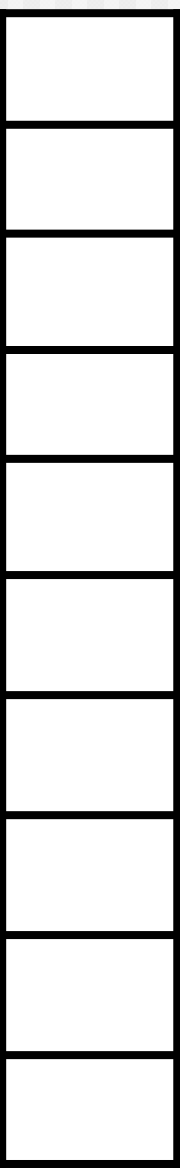
```
  return 0;
```

```
}
```

```
void fun(int *arr,int n)
```

```
{ | }
```

arr →  } array[0]  
arr[0]

arr+3 →  } array[3]  
arr[3]

array数组

# 形参数组名与实参数组名的区别

- 实参数组名是指针常量，但形参数组名是按指针变量处理
- 在函数调用实参传给形参后，它的值就是实参数组首元素的地址
- 在函数执行期间，形参数组名可以再被赋值

```
void fun (int arr[ ],int n)
```

```
{ printf("%d\n", *arr);
```

```
    arr=arr+3; //不会改变实参数组的首地址
```

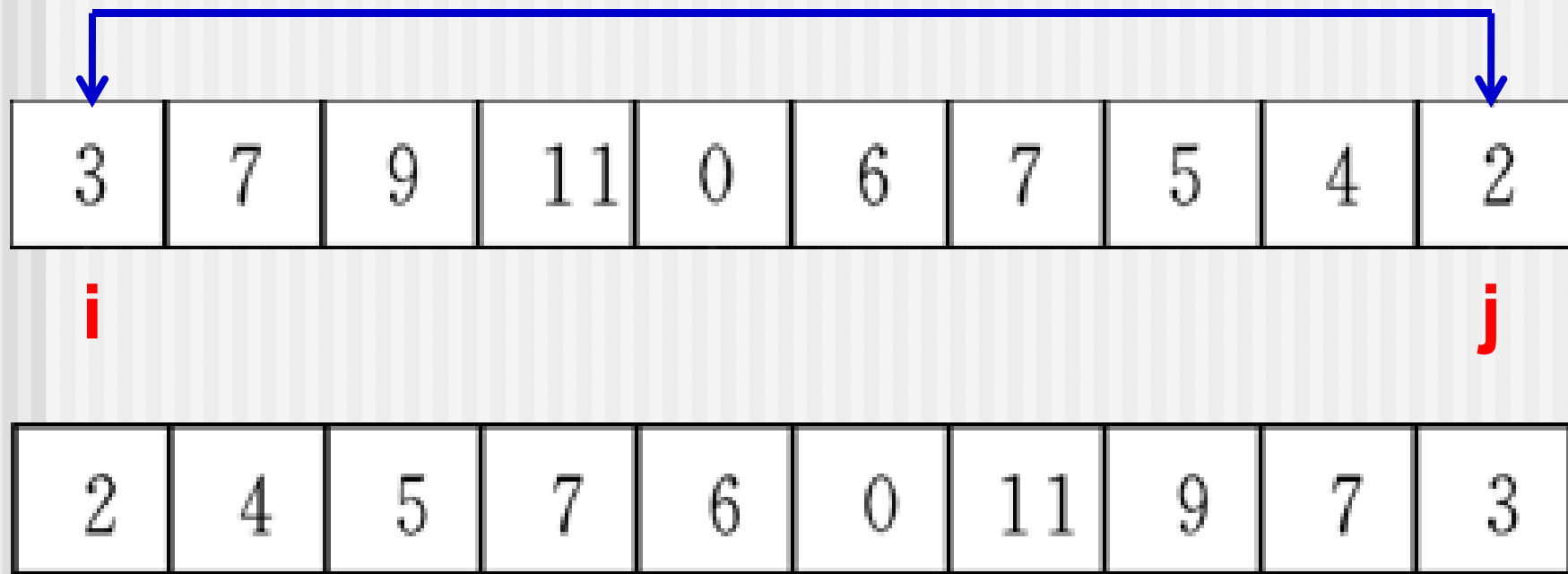
```
    printf("%d\n", *arr);
```

```
}
```



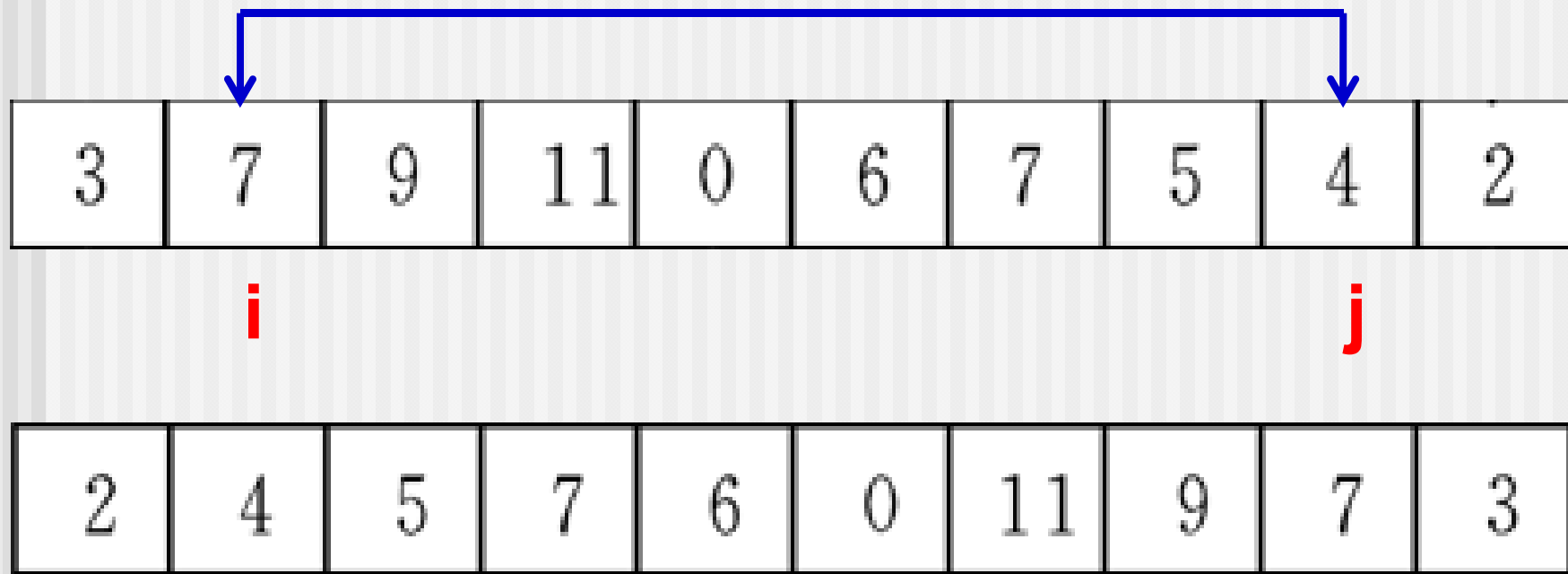
## 【例8.8】 将数组a中n个整数按相反顺序存放

- 解题思路：将 $a[0]$ 与 $a[n-1]$ 对换，.....将 $a[4]$ 与 $a[5]$ 对换。



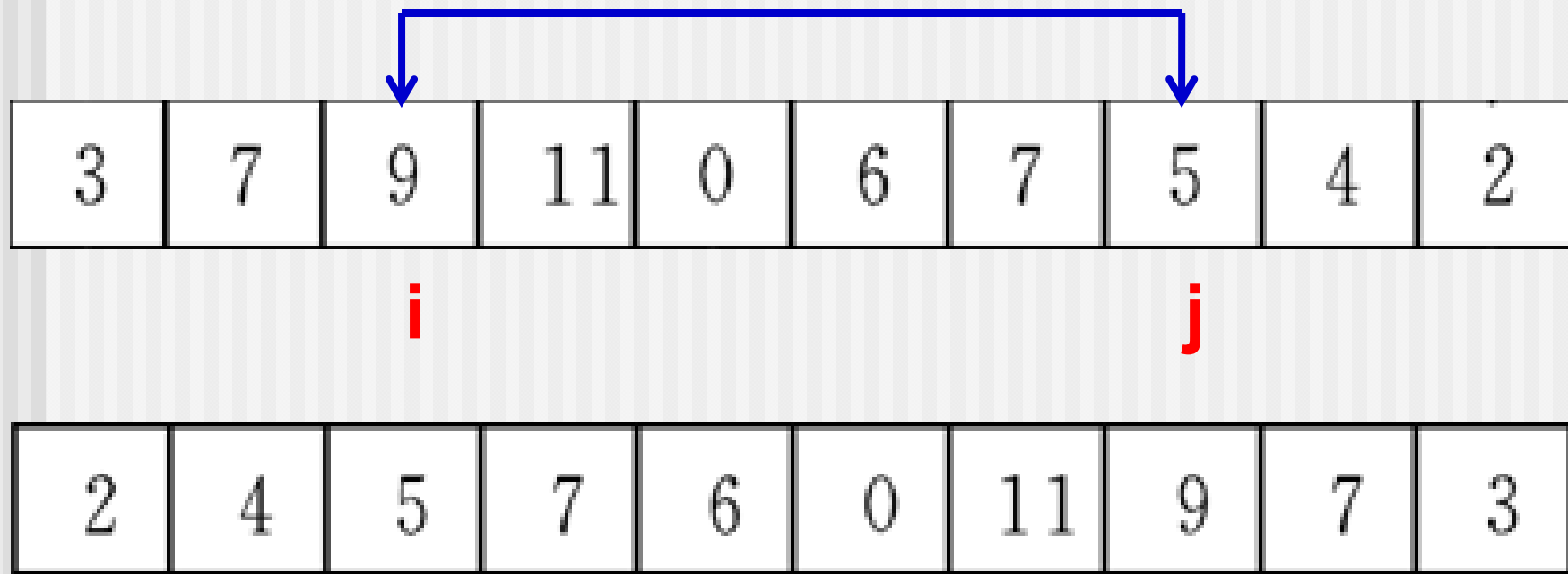
## 【例8.8】 将数组a中n个整数按相反顺序存放

- 解题思路：将 $a[0]$ 与 $a[n-1]$ 对换，.....将 $a[4]$ 与 $a[5]$ 对换。



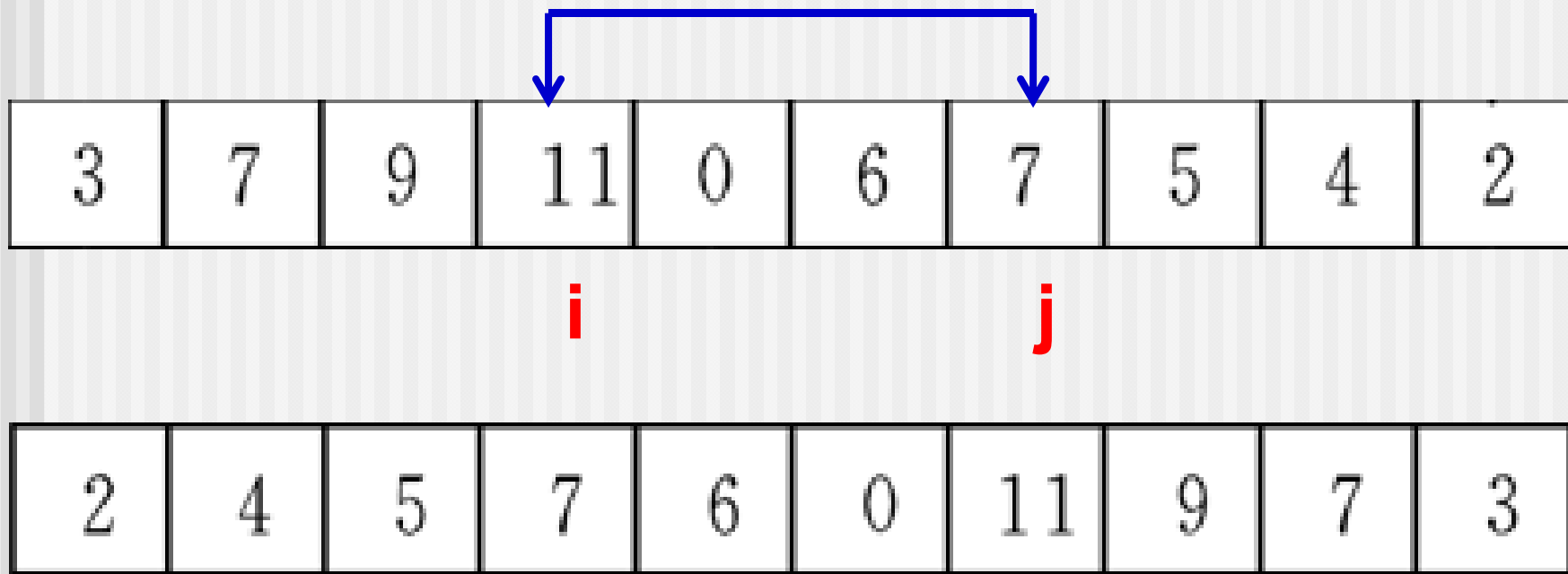
## 【例8.8】 将数组a中n个整数按相反顺序存放

- 解题思路：将 $a[0]$ 与 $a[n-1]$ 对换，.....将 $a[4]$ 与 $a[5]$ 对换。



## 【例8.8】 将数组a中n个整数按相反顺序存放

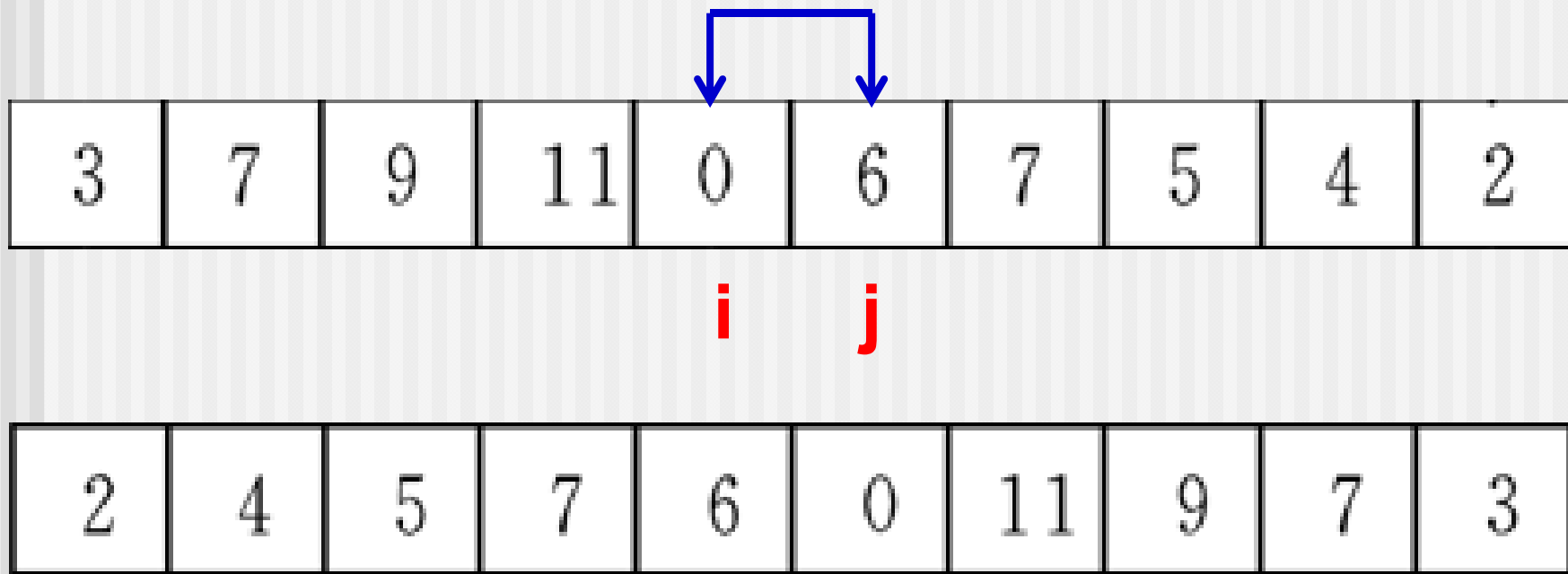
- 解题思路：将 $a[0]$ 与 $a[n-1]$ 对换，.....将 $a[4]$ 与 $a[5]$ 对换。





## 【例8.8】 将数组a中n个整数按相反顺序存放

- 解题思路：将 $a[0]$ 与 $a[n-1]$ 对换，.....将 $a[4]$ 与 $a[5]$ 对换。



```
#include <stdio.h>
int main()
{ void inv(int x[ ],int n);
  int i, a[10]={3,7,9,11,0,6,7,5,4,2};
  for(i=0;i<10;i++) printf("%d ",a[i]);
  printf("\n");
  inv(a,10);
  for(i=0;i<10;i++) printf("%d ",a[i]);
  printf("\n");
  return 0;
}
```

```
void inv(int x[ ],int n)
{ int temp,i,j,m=(n-1)/2;
  for(i=0;i<=m;i++)
  { j=n-1-i;
    temp=x[i];x[i]=x[j];x[j]=temp;
  }
}
```

优化

```
void inv(int x[ ],int n)
{ int temp,*i,*j;
  i=x; j=x+n-1;
  for( ; i<j; i++,j--)
  { temp=*i; *i=*j; *j=temp; }
}
```

例8.9 改写例8.8，用指针变量作实参。

```
#include <stdio.h>
```

```
int main()
```

```
{ void inv(int *x,int n);
```

```
int i, arr[10], *p=arr;
```

```
for(i=0;i<10;i++,p++)
```

```
scanf("%d",p);
```

```
p=arr; inv(p,10);
```

```
for(p=arr;p<arr+10;p++)
```

```
printf("%d ",*p);
```

```
printf("\n");
```

```
return 0;
```

```
}
```

不可少!!!

不可少!!!

可以省略

## 例8.10 用指针方法对10个整数按由大到小顺序排序。

---

### ■ 解题思路：

- 在主函数中定义数组a存放10个整数，定义int \*型指针变量p指向a[0]
- 定义函数sort使数组a中的元素按由大到小的顺序排列
- 在主函数中调用sort函数，用指针p作实参
- 用选择法进行排序

```
#include <stdio.h>
int main()
{ void sort(int x[ ],int n);
  int i,*p,a[10];
  p=a;
  for(i=0;i<10;i++) scanf("%d",p++);
  p=a;
  sort(p,10);
  for(p=a,i=0;i<10;i++)
  { printf("%d ",*p); p++; }
  printf("\n");
  return 0;
}
```

**printf("%d ",\*p++);**



**void sort(int \*x,int n)**

**void sort(int x[],int n)**

{ int i,j,k,t;

for(i=0;i<n-1;i++)

{ k=i;

**if (\*(x+j)>\*(x+k)) k=j;**

for(j=i+1;j<n;j++)

**if(x[j]>x[k]) k=j;**

if(k!=i)

**{ t=x[i];x[i]=x[k];x[k]=t; }**

**{t=\*(x+i);\*(x+i)=\*(x+k);\*(x+k)=t;}**

12 34 5 689 -43 56 -21 0 24 65  
689 65 56 34 24 12 5 0 -21 -43

## 8.4 通过指针引用字符串

---

8.4.1 字符串的引用方式

8.4.2 字符指针作函数参数

8.4.3 使用字符指针变量和  
字符数组的比较

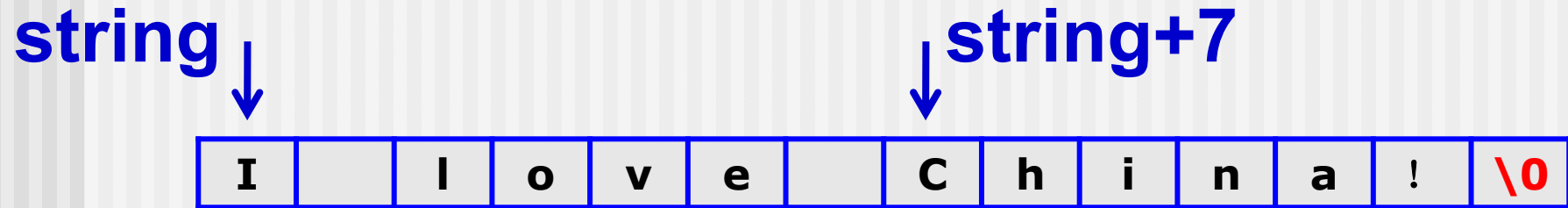
## 8.4.1 字符串的引用方式

- 字符串是存放在字符数组中的。引用一个字符串，可以用以下两种方法。
  - (1) 用字符数组存放一个字符串，可以通过数组名和格式声明“%s”输出该字符串，也可以通过数组名和下标引用字符串中一个字符。
  - (2) 用字符指针变量指向一个字符串常量，通过字符指针变量引用字符串常量。
- C在内存中为每个字符串常量分配一块连续的存储区（逻辑结构上与字符数组相同，但所存储的内容不能修改）

**【例8.16】** 定义一个字符数组，在其中存放字符串“I love China!”，输出该字符串和第8个字符。

---

- 解题思路：定义字符数组**string**，对它初始化，由于在初始化时字符的个数是确定的，因此可不必指定数组的长度。用数组名**string**和输出格式**%s**可以输出整个字符串。用数组名和下标可以引用任一数组元素。



```
#include <stdio.h>
int main()
{ char string[]="I love China!";
  printf("%s\n", string);
  printf("%c\n", string[7]);
  return 0;
}
```

```
I love China!
C
```

## 【例8.17】通过字符指针变量输出一个字符串。

---

- 解题思路：可以不定义字符数组，只定义一个字符指针变量，用它指向字符串常量中的字符。通过字符指针变量输出该字符串。



常量存储区

|   |  |   |   |   |   |  |   |   |   |   |   |   |    |
|---|--|---|---|---|---|--|---|---|---|---|---|---|----|
| I |  | l | o | v | e |  | C | h | i | n | a | ! | \0 |
|---|--|---|---|---|---|--|---|---|---|---|---|---|----|

动态存储区

string

```
#include <stdio.h>
```

```
int main()
```

```
{ char *string="I love China!";
```

```
printf("%s\n",string);
```

```
return 0;
```

```
}
```

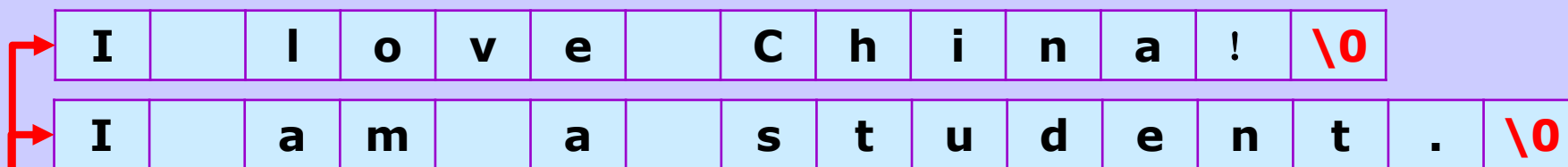
```
char *string;
```

```
string=" I love China!";
```

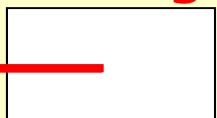
```
I love China!
```

存放的是字符串常量的首地址，  
而不是字符串本身。

常量存储区



动态存储区 **string**



```
#include <stdio.h>
```

```
int main()
```

```
{ char *string="I love China!";
```

```
printf("%s\n",string);
```

```
string="I am a student.";
```

```
printf("%s\n",string);
```

```
return 0;
```

```
}
```

```
I love China!
I am a student.
```

**【例8.18】**将字符串**a**复制到字符串**b**，然后输出字符串**b**。

---

- 解题思路：定义两个字符数组**a**和**b**，用“**I am a student.**”对**a**数组初始化。将**a**数组中的字符逐个复制到**b**数组中。可以用不同的方法引用并输出字符数组元素，今用地址法算出各元素的值。

**a** ↓

|   |  |   |   |  |   |  |   |   |   |   |   |   |   |   |    |
|---|--|---|---|--|---|--|---|---|---|---|---|---|---|---|----|
| I |  | a | m |  | a |  | s | t | u | d | e | n | t | . | \0 |
|---|--|---|---|--|---|--|---|---|---|---|---|---|---|---|----|

**b** ↓

|   |  |   |   |  |   |  |   |   |   |   |   |   |   |   |    |  |  |  |  |
|---|--|---|---|--|---|--|---|---|---|---|---|---|---|---|----|--|--|--|--|
| I |  | a | m |  | a |  | s | t | u | d | e | n | t | . | \0 |  |  |  |  |
|---|--|---|---|--|---|--|---|---|---|---|---|---|---|---|----|--|--|--|--|

```
#include <stdio.h>
```

```
int main()
```

```
{ char a[ ]="I am a stu
```

```
int i;
```

```
for(i=0;*(a+i)!='\0';i++)
```

```
    *(string a is:I am a student.
```

```
    *(b+string b is:I am a student.
```

```
printf("string a is:%s\n",a);
```

```
printf("string b is:");
```

```
for(i=0;b[i]!='\0';i++)
```

```
    printf("%c",b[i]);
```

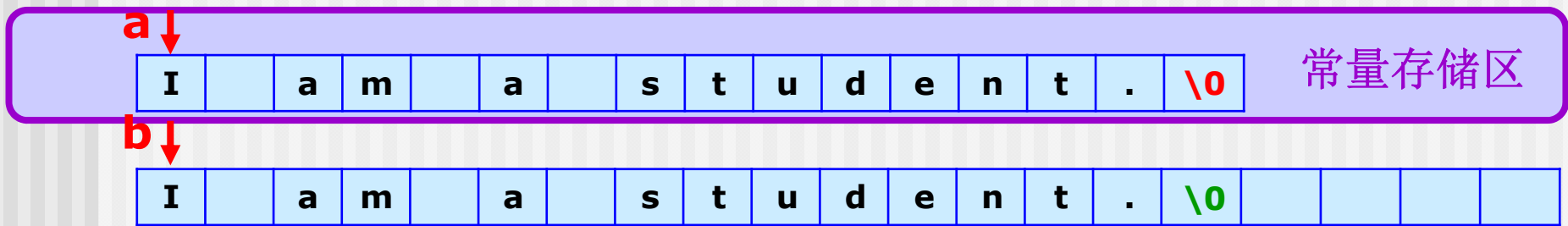
```
printf("\n");
```

```
return 0;
```

```
}
```

for(i=0; a[i]!='\0';i++)  
b[i]=a[i];  
b[i]='\0';

printf("string b is:%s\n",b);



```
#include <stdio.h>
char *a="I am a student.",b[20];
int main()
{ char a[ ]="I am a student.",b[20];
  int i;
  for(i=0;*(a+i)!='\0';i++)
    *(b+i)=*(a+i);
  *(b+i)='\0';
  printf("string a is:%s",a);
  printf("string b is:");
  for(i=0;b[i]!='\0';i++)
    printf("%c",b[i]);
  printf("\n");
  return 0;
}
```

for(i=0; a[i]!='\0';i++)  
b[i]=a[i];  
b[i]='\0';

for(i=0; \*a!='\0';i++)  
b[i]=\*a++;  
b[i]='\0';

## 【例8.19】 用指针变量来处理例8.18问题。

---

- 解题思路：定义两个指针变量**p1**和**p2**，分别指向字符数组**a**和**b**。改变指针变量**p1**和**p2**的值，使它们顺序指向数组中的各元素，进行对应元素的复制。



a ↓↓ p1

|   |  |   |   |  |   |  |   |   |   |   |   |   |   |   |    |
|---|--|---|---|--|---|--|---|---|---|---|---|---|---|---|----|
| I |  | a | m |  | a |  | s | t | u | d | e | n | t | . | \0 |
|---|--|---|---|--|---|--|---|---|---|---|---|---|---|---|----|

b ↓↓ p2

|   |  |   |   |  |   |  |   |   |   |   |   |   |   |   |    |  |  |  |  |
|---|--|---|---|--|---|--|---|---|---|---|---|---|---|---|----|--|--|--|--|
| I |  | a | m |  | a |  | s | t | u | d | e | n | t | . | \0 |  |  |  |  |
|---|--|---|---|--|---|--|---|---|---|---|---|---|---|---|----|--|--|--|--|

```
# for(p1=a,p2=b; (*p2++=*p1++)!='\0';)
```

```
int main()  
{  
    for(p1=a,p2=b; (*p2=*p1)!='\0';p1++,p2++)
```

```
    p1=a; p2=b;
```

```
    for( ; *p1!='\0'; p1++,p2++)
```

```
        *p2=*p1;
```

```
    *p2='\0';
```

```
string a is:I am a student.
```

```
string b is:I am a student.
```

```
    printf("string a is:%s\n",a);
```

```
    printf("string b is:%s\n",b);
```

```
    return 0;
```

```
}
```

## 8.4.2 字符指针作函数参数

- 如果想把一个字符串从一个函数“传递”到另一个函数，可以用地址传递的办法，即用字符数组名作参数，也可以用字符指针变量作参数。
- 在被调用的函数中可以改变字符串的内容
- 在主调函数中可以引用改变后的字符串。

## 8.4.2 字符指针作函数参数

【例8.20】 用函数调用实现字符串的复制。

- 解题思路：定义一个函数`copy_string`用来实现字符串复制的功能，在主函数中调用此函数，函数的形参和实参可以分别用字符数组名或字符指针变量。分别编程，以供分析比较。

## (1) 用字符数组名作为函数参数

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    void copy_string(char from[],char to[]);
```

```
    char a[]="I am a teacher.";
```

```
    char b[]="you are a student.";
```

```
    printf("a=%s\nb=%s\n",a,b);
```

```
    printf("copy string a to string b:\n");
```

```
    copy_string(a,b);
```

```
    printf("a=%s\nb=%s\n",a,b);
```

```
    return 0;
```

```
}
```

main函数

a ↓  
I a m a t e a c h e r . \0

b ↓  
I a m a t e a c h e r . \0 t . \0

to

from

copy\_string函数

```
void copy_string(char from[], char to[])  
{ int i=0;  
  while(from[i]!='\0')  
  { to[i]=from[i];  
    i++;  
  }  
  to[i]='\0';  
}
```

```
a=I am a teacher.  
b=You are a student.  
copy string a to string b:  
a=I am a teacher.  
b=I am a teacher.
```

## (2)用字符型指针变量作实参

- `copy_string`不变，在`main`函数中定义字符指针变量`from`和`to`，分别指向两个字符数组`a`,`b`。
- 仅需要修改主函数代码

```
#include <stdio.h>
int main()
{void copy_string(char from[], char to[]);
 char a[]="I am a teacher.";
 char b[]="you are a student.";
 char *from=a,*to=b;
 printf("a=%s\nb=%s\n",a,b);
 printf("\ncopy string a to string b:\n");
 copy_string(from,to);
 printf("a=%s\nb=%s\n",a,b);
 return 0;
}
```



### (3)用字符指针变量作形参和实参

```
#include <stdio.h>
```

```
int main()
```

```
{void copy_string(char *from, char *to);
```

```
    char *a="I am a teacher.";
```

```
    char b[]="You are a student.";
```

```
    char *p=b;
```

```
    printf("a=%s\nb=%s\n",a,b);
```

```
    printf("\ncopy string a to string b:\n");
```

```
    copy_s
```

```
    printf("\n
```

```
    return 0;
```

```
}
```

函数体有多种简化写法，请见教材**P262**

```
void copy_string(char *from, char *to)
```

```
{ for( ;*from!='\0'; from++,to++)
```

```
    { *to=*from; }
```

```
    *to='\0';
```

```
}
```

## 【例8.20】用函数调用实现字符串的复制。

### (2) 字符指针变量作形参

```
void copy_string(char *from, char *to)
```

```
{ for (; *from != '\0'; from++, to++)
```

①        \*to = \*from;

         \*to = '\0';

```
}
```

② { while (\*from != '\0') \*to++ = \*from++; \*to = '\0'; }

③ { while ((\*to = \*from) != '\0') { from++; to++; } }

④ { while ((\*to++ = \*from++) != '\0'); }

⑤ { while (\*to++ = \*from++); }

⑥ { for (; \*to++ = \*from++; ); }

## 8.4.3 使用字符指针变量和字符数组的比较

- 用字符数组和字符指针变量都能实现字符串的存储和运算，但它们二者之间是有区别的，不应混为一谈，主要有以下几点。

(1) **字符数组**由若干个元素组成，每个元素中放一个字符，而**字符指针变量**中存放的是地址（字符串第1个字符的地址），绝不是将字符串放到字符指针变量中。

(2) **赋值方式**。可以对字符指针变量赋值，但不能对数组名赋值。

`char *a; a="I love China!";` 对

`char str[14];str[0]='I';` 对

`char str[14]; str="I love China!";` 错

### 8.4.3 使用字符指针变量和字符数组的比较

- 用字符数组和字符指针变量都能实现字符串的存储和运算，但它们二者之间是有区别的，不应混为一谈，主要有以下几点。

(3) 初始化的含义

`char *a="I love China! ";`与

`char *a; a="I love China! ";`等价

对字符数组的初始化:

`char str[14]="I love China! ";`与

`char str[14];`

`str[]="I love China! ";` 不等价，错误！

- 对字符数组只能逐个元素赋值！

### 8.4.3 使用字符指针变量和字符数组的比较

- 用字符数组和字符指针变量都能实现字符串的存储和运算，但它们二者之间是有区别的，不应混为一谈，主要有以下几点。

#### (4) 存储单元的内容

编译时为字符数组分配若干存储单元，以存放各元素的值，而对字符指针变量，只分配一个存储单元。

```
char *a; scanf("%s",a); 错
```

```
char a,str[10];
```

```
a=str;
```

```
scanf ("%s",a); 对
```



(5) 指针变量的值是可以改变的，而数组名代表一个固定的值(数组首元素的地址)，不能改变。

【例8.21】

```
#include <stdio.h>
```

```
int main()
```

```
{ char *a="I love China!";
```

```
  a=a+7;
```

```
  printf("%s\n",a);
```

```
  return 0;
```

```
}
```

不能改为

**char a[]="I love China!";**

China!



### 8.4.3 使用字符指针变量和字符数组的比较

- 用字符数组和字符指针变量都能实现字符串的存储和运算，但它们二者之间是有区别的，不应混为一谈，主要有以下几点。

(6) 字符数组中各元素的值是可以改变的，但字符指针变量指向的字符串常量中的内容是不可以被取代的。

```
char a[]="House",*b=" House";
```

```
a[2]='r';          对      *(b+2)='r';  错!
```

(7) 引用数组元素

对字符数组可以用下标法和地址法引用数组元素

(`a[5]`, `*(a+5)`)。如果字符指针变量 `p=a`，则也可以用指针变量带下标的形式和地址法引用

(`p[5]`, `*(p+5)`)。

```
char *a="I love China!";
```

则 `a[5]` 的值是第6个字符，即字母 'e'

### 8.4.3 使用字符指针变量和字符数组的比较

用字符数组和字符指针变量都能实现字符串的存储和运算，但它们二者之间是有区别的，不应混为一谈，主要有以下几点。

(8) 用指针变量指向一个格式字符串，可以用它代替printf函数中的格式字符串。

```
char *format;
```

```
format="a=%d,b=%f\n";
```

```
printf(format,a,b);
```

相当于

```
printf("a=%d,b=%f\n",a,b);
```

## 8.8 动态内存分配与指向它的 指针变量

---

8.8.1 什么是内存的动态分配

8.8.2 怎样建立内存的动态分配

8.8.3 void指针类型

## 8.8.1 什么是内存的动态分配

---

- 非静态的局部变量是分配在内存中的动态存储区的，这个存储区是一个称为**栈**的区域
- C语言还允许建立内存动态分配区域，以存放一些临时用的数据，这些数据需要时随时开辟，不需要时随时释放。这些数据是临时存放在一个特别的自由存储区，称为**堆**区

## 8.8.2 怎样建立内存的动态分配

- 对内存的动态分配是通过系统提供的库函数来实现的，主要有**malloc**，**calloc**，**free**，**realloc**这4个函数。

### 1 . **malloc**函数

- 函数原型：

**void \*malloc(unsigned int size);**

- 其作用是在内存的动态存储区中分配一个长度为**size**的连续空间
- 函数的值是为所分配区域的第一个字节的地址，或者说，此函数是一个指针型函数，返回的指针指向该分配域的开头位置



## 8.8.2 怎样建立内存的动态分配

---

`malloc(100);`

- 开辟100字节的临时分配域，函数值为其第1个字节的地址
- 注意指针的基类型为**void**，即不指向任何类型的数据，只提供一个地址
- 如果此函数未能成功地执行（例如内存空间不足），则返回空指针(NULL)



## 8.8.2 怎样建立内存的动态分配

---

### 2. **calloc**函数

- 其函数原型为

`void *calloc(unsigned n,unsigned size);`

- 其作用是在内存的动态存储区中分配n个长度为**size**的连续空间，这个空间一般比较大，足以保存一个数组。

## 8.8.2 怎样建立内存的动态分配

- 用**calloc**函数可以为二维数组开辟动态存储空间，**n**为数组元素个数，每个元素长度为**size**。这就是动态数组。函数返回指向所分配域的起始位置的指针；如果分配不成功，返回**NULL**。如：

**p=calloc(50,4);**

开辟**50×4**个字节的临时分配域，把起始地址赋给指针变量**p**

## 8.8.2 怎样建立内存的动态分配

---

### 3. **free**函数

- 其函数原型为

**void free(void \*p);**

- 其作用是释放指针变量 p 所指向的动态空间，使这部分空间能重新被其他变量使用。p 应是最近一次调用 **calloc** 或 **malloc** 函数时得到的函数返回值。

## 8.8.2 怎样建立内存的动态分配

---

`free(p);`

- 释放指针变量 `p` 所指向的已分配的动态空间
- `free` 函数无返回值

## 8.8.2 怎样建立内存的动态分配

---

### 4. **realloc**函数

- 其函数原型为

`void *realloc(void *p, unsigned int size);`

- 如果已经通过**malloc**函数或**calloc**函数获得了动态空间，想改变其大小，可以用**realloc**函数重新分配。

## 8.8.2 怎样建立内存的动态分配

- 用**realloc**函数将**p**所指向的动态空间的大小改变为**size**，若新空间比原空间大，则原有数据保持不变。如果重分配不成功，返回**NULL**。如

**realloc(p,50);**

将**p**所指向的已分配的动态空间改为**50**字节。

以上4个函数的声明在**stdlib.h**头文件中，在用到这些函数时应当用“**#include <stdlib.h>**”指令把**stdlib.h**头文件包含到程序文件中。



## 8.8.3 void指针类型

**【例8.30】** 建立动态数组，输入5个学生的成绩，另外用一个函数检查其中有无低于60分的，输出不合格的成绩。

**【解题思路】** 用**malloc**函数开辟一个动态自由区域，用来存5个学生的成绩，会得到这个动态域第一个字节的地址，它的基类型是**void**型。用一个基类型为**int**的指针变量**p**来指向动态数组的各元素，并输出它们的值。但必须先把**malloc**函数返回的**void**指针转换为整型指针，然后赋给**p1**

```
#include <stdio.h>
#include <stdlib.h>
int main()
{ void check(int *);
  int *p1,i;
  p1=(int *)malloc(5*sizeof(int));
  for(i=0;i<5;i++)
    scanf("%d",p1+i);
  check(p1);
  return 0;
}
```

---

```
void check(int *p)
{ int i;
  printf("They are fail:");
  for(i=0;i<5;i++)
    if (p[i]<60)
      printf("%d ",p[i]);
  printf("\n");
}
```

```
67 98 59 78 57
They are fail:59 57
```

## 8.9有关指针的小结

---

1. 首先要准确地弄清楚指针的含义。指针就是地址，凡是出现“指针”的地方，都可以用“地址”代替，例如，变量的指针就是变量的地址，指针变量就是地址变量
- 要**区别指针和指针变量**。指针就是地址本身，而指针变量是用来存放地址的变量。

## 8.9有关指针的小结

---

2. 什么叫“指向”？地址就意味着指向，因为通过地址能找到具有该地址的对象。对于指针变量来说，把谁的地址存放在指针变量中，就说此指针变量指向谁。但应注意：只有与指针变量的基类型相同的数据的地址才能存放在相应的指针变量中。



## 8.9 有关指针的小结

**void \***指针是一种特殊的指针，所指向的内存单元数据类型不确定，如果需要访问该指针所指向的数据，应先对其进行类型转换。可以在程序中进行显式的类型转换，也可以由编译系统自动进行隐式转换。无论用哪种转换，读者必须了解要进行类型转换

例如：`int a; void *p=a;`  
`*(int *) p=3;`



## 8.9有关指针的小结

3. 要深入掌握在对数组的操作中怎样正确地使用指针，搞清楚指针的指向。一维数组名代表数组首元素的地址

```
int *p,a[10];  
p=a;
```

- **p**是指向**int**类型的指针变量，**p**只能指向数组中的元素，而不是指向整个数组。在进行赋值时一定要先确定赋值号两侧的类型是否相同，是否允许赋值。
- 对“**p=a;**”，准确地说应该是：**p**指向**a**数组的首元素

## 8.9 有关指针的小结

4. 有关指针变量的定义形式的归纳比较, 见主教材 P289 表 8.4。

| 变量定义                       | 类型表示                     | 含 义                                                            |
|----------------------------|--------------------------|----------------------------------------------------------------|
| <code>int i;</code>        | <code>int</code>         | 定义整型变量 <code>i</code>                                          |
| <code>int * p;</code>      | <code>int *</code>       | 定义 <code>p</code> 为指向整型数据的指针变量                                 |
| <code>int a[5]</code>      | <code>int [5]</code>     | 定义整型数组 <code>a</code> , 它有 5 个元素                               |
| <code>int * p[4];</code>   | <code>int * [4]</code>   | 定义指针数组 <code>p</code> , 它由 4 个指向整型数据的指针元素组成                    |
| <code>int (* p)[4];</code> | <code>int (*) [4]</code> | <code>p</code> 为指向包含 4 个元素的一维数组的指针变量                           |
| <code>int f();</code>      | <code>int ()</code>      | <code>f</code> 为返回整型函数值的函数                                     |
| <code>int * p();</code>    | <code>int * ()</code>    | <code>p</code> 为返回一个指针的函数, 该指针指向整型数据                           |
| <code>int (* p)();</code>  | <code>int (*) ()</code>  | <code>p</code> 为指向函数的指针, 该函数返回一个整型值                            |
| <code>int ** p;</code>     | <code>int **</code>      | <code>p</code> 是一个指针变量, 它指向一个指向整型数据的指针变量                       |
| <code>void * p;</code>     | <code>void *</code>      | <code>p</code> 是一个指针变量, 基类型为 <code>void</code> (空类型), 不指向具体的对象 |

## 8.9 有关指针的小结

### 5. 指针运算

#### (1) 指针变量加（减）一个整数

例如： $p++$ ,  $p--$ ,  $p+i$ ,  $p-i$ ,  $p+=i$ ,  $p-=i$  等均是指针变量加（减）一个整数。

- 将该指针变量的原值(是一个地址)和它指向的变量所占用的存储单元的字节数相加（减）。

#### (2) 指针变量赋值

- 将一个变量地址赋给一个指针变量
- 不应把一个整数赋给指针变量

#### (3) 两个指针变量可以相减

- 如果两个指针变量都指向同一个数组中的元素，则两个指针变量值之差是两个指针之间的元素个数。

## 8.9有关指针的小结

### 5. 指针运算

#### (4) 两个指针变量比较

- 若两个指针指向同一个数组的元素，则可以进行比较
- 指向前面的元素的指针变量“小于”指向后面元素的指针变量
- 如果p1和p2不指向同一数组则比较无意义

(5) 指针变量可以有空值，即该指针变量不指向任何变量，可以这样表示：

`p=NULL;`

# 作业

---

- Part A
  - P291 8.4, 8.5;
- Part B
  - P291 8.7;
  - P292 8.16, 8.18;
- Part C
  - P292 8.19;